

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\agent-a611f50714ff43867.jsonl`  
- SHA-256: `2f96e94413e660f4ed45edaaa19f776596a64472a24ca91bea8c9b1a69e05089`  
- Source modified: `2026-06-14T09:53:13+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `subagents`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-14T09:49:43.245Z)

Design a reusable ABLATION LAYER / FRAMEWORK for the rally-detection system and WRITE the design to scratch/ABLATION_FRAMEWORK_PLAN.md (you may write to scratch/ only; read-only on tracked code). Owner's intent: a framework which, when run, EXCLUDES a single signal and measures whether quality dropped ? so over the EVOLUTION of quality we can assess, QUANTIFIABLY, whether older signals are still relevant or have become dead weight. The systematic, repeatable version of a one-off ablation; it underpins the paper-coauthor aim (the ablation table IS the empirical backbone).

In the badminton-highlight-indexer repo (cwd), READ: backend/eval/experiment.py (Variant/compare/LOVO/leaderboard/record_experiment), backend/eval/eval_stats.py (bootstrap CI + sign test + grouped_folds + out_of_fold_predictions), backend/pipeline/detectors/fusion_features.py + backend/pipeline/detectors/audio_features.py (signals as feature columns), backend/eval/per_user_gate.py (promotion logic to compose with), docs/EXPERIMENT_HARNESS.md + docs/ANALYZERS_AND_RECONCILERS.md (the signal-fusion framing), docs/QUALITY_ITERATIONS.md (the living quality log this feeds).

DESIGN: (1) a leave-one-signal-out (and leave-one-GROUP-out) ablation RUNNER over a corpus that, for each signal/group, reports the marginal ?F1 (CI + sign test) when removed from the full set = the signal's MARGINAL CONTRIBUTION; (2) a SIGNAL-RELEVANCE LEDGER tracking each signal's marginal contribution ACROSS quality iterations over TIME (so a signal that stops earning its keep is flagged quantifiably ? "is this old signal still relevant?"); (3) REDUNDANCY/INTERACTION handling ? two signals each useless alone but useful together (and vice versa) where naive leave-one-out misleads ? note leave-PAIR-out / interaction / a backward-elimination or Shapley-style attribution and the cost trade-off; (4) honest small-N handling (reuse eval_stats; don't over-claim at n=6); (5) integration with the experiment harness + leaderboard + QUALITY_ITERATIONS; (6) how it generalizes to per-user ablation (ties to the personalization gate ? "motion cue not relevant FOR THIS USER"). Include a PHASED PR-sized build plan (Phase 0 = the leave-one-signal-out runner over the golden set reusing the harness + a results table; later = the time-series relevance ledger + interaction analysis) marking engineering vs owner-gated.

Return: a ~250-word exec summary + the Phase-0 PR list + the top open design questions. Write the full design to scratch/ABLATION_FRAMEWORK_PLAN.md.

2. assistant (2026-06-14T09:49:45.545Z)

I'll start by reading the key files to understand the existing architecture, then design the ablation framework.

3. user (2026-06-14T09:49:48.571Z)

Already up to date.

4. user (2026-06-14T09:49:48.584Z)

```
1      """Honest small-N evaluation statistics (course-corrections C1 + C3).
```

```

2
3     The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
a
4     single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
training
5     and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
6     ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
7     turns those into an honest summary:
8
9     - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
10     interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
11     many *videos* it wins on (the distribution-free test the repo already leans on:
12     "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
noise.
13     - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
learned
14     producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
15     match, never a held-out window from the same video).
16
17     See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
only),
18     deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
19     ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
20     ""
21     from __future__ import annotations
22
23     import math
24     import random
25     import statistics
26     from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
27
28     Interval = Tuple[float, float]
29     FitPredict = Callable[[List[int], List[int]], List[Any]]
30
31
32     def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
33                     n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
34         """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
35
36         ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
to
37         resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
band.
38         ""
39         vals = [float(v) for v in values]
40         n = len(vals)
41         if n == 0:
42             return (0.0, 0.0)
43         if n == 1:
44             return (vals[0], vals[0])
45         rng = random.Random(seed)
46         means: List[float] = []
47         for _ in range(max(1, n_boot)):
48             means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
49         means.sort()
50         alpha = (1.0 - confidence) / 2.0
51         last = len(means) - 1
52         lo = means[max(0, int(math.floor(alpha * last)))]
53         hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
54         return (lo, hi)
55
56
57     def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
58                     n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
59         """"{mean, std, n, ci_low, ci_high, confidence}"" ? report this, never a bare mean

```

```

(C1)."""
60     vals = [float(v) for v in values]
61     n = len(vals)
62     mean = statistics.mean(vals) if vals else 0.0
63     std = statistics.pstdev(vals) if n > 1 else 0.0
64     lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
65     return {"mean": mean, "std": std, "n": float(n),
66            "ci_low": lo, "ci_high": hi, "confidence": confidence}
67
68
69 def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
70     """Two-sided exact sign test over per-fold deltas (B ? A).
71
72     ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
73     excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
74     distribution-free "did B win on enough *videos*" check.
75     """
76     wins = sum(1 for d in deltas if d > eps)
77     losses = sum(1 for d in deltas if d < -eps)
78     ties = len(deltas) - wins - losses
79     n_eff = wins + losses
80     if n_eff == 0:
81         p = 1.0
82     else:
83         k = min(wins, losses)
84         tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
85         p = min(1.0, 2.0 * tail)
86     return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
87            "n_effective": float(n_eff), "p_value": p}
88
89
90 def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
91                    confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
92                    eps: float = 0.0) -> Dict[str, Any]:
93     """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).
94
95     ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
96     by
97     fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
98     sign
99     test agrees ? not when a single mean ticked up.
100     """
101     n = min(len(a_values), len(b_values))
102     deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
103     summary = mean_with_ci(deltas, confidence, n_boot, seed)
104     return {
105         "mean_delta": summary["mean"],
106         "ci_low": summary["ci_low"],
107         "ci_high": summary["ci_high"],
108         "n": float(n),
109         "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
110         "sign_test": paired_sign_test(deltas, eps),
111     }
112
113 def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
114     """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
115     unit.
116
117     The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
118     first-seen (deterministic). Groups with no complement (a single group) yield no
119     fold.
120
121     """
122     seen: List[Any] = []
123     for g in groups:

```

```

120         if g not in seen:
121             seen.append(g)
122         n = len(groups)
123         folds: List[Tuple[List[int], List[int]]] = []
124         for g in seen:
125             test = [i for i in range(n) if groups[i] == g]
126             train = [i for i in range(n) if groups[i] != g]
127             if test and train:
128                 folds.append((train, test))
129         return folds
130
131
132     def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
133         """Out-of-fold predictions for honest stacking (C3).
134
135         For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
136         the
137         other groups and returns predictions for ``test_idx`` (in that order); results are
138         scattered back to a list aligned to the input items. So a *learned* producer's
139         outputs,
140         when used as arbiter features, never include in-sample predictions ? the leakage
141         path
142         that inflates measured lift on a tiny corpus. Items in a singleton group stay
143         ``None``
144         (needs >= 2 groups to predict).
145         """
146         n = len(groups)
147         oof: List[Optional[Any]] = [None] * n
148         for train_idx, test_idx in grouped_folds(groups):
149             preds = fit_predict(train_idx, test_idx)
150             for pos, ti in enumerate(test_idx):
151                 if pos < len(preds):
152                     oof[ti] = preds[pos]
153         return oof

```

5. user (2026-06-14T09:49:49.062Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. **No new scoring math lives
7     here**; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23
24    Three modes (`EXPERIMENT_HARNESS.md` §5):
25    - `compare()`      ? labeled A/B vs golden labels: per-video + LOVO-honest
26    aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-

```

```

27     held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import json
41     import logging
42     import os
43     from dataclasses import dataclass, field
44     from datetime import datetime, timezone
45     from hashlib import sha1
46     from typing import Any, Callable, Dict, List, Optional, Sequence
47
48     from backend.eval.calibration import pct_improvement
49     from backend.eval.classifier import LogisticRegression
50     from backend.eval.gemini_refine import merge_overlaps
51     from backend.eval.metrics import Interval, match_intervals, score
52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
56 ratio
57 scores
58     from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
59
60     logger = logging.getLogger(__name__)
61
62     # Where a comparison run appends one reproducible record. Tracked location (NOT under
63     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
64     # regression.DEFAULT_BASELINE_PATH.
65     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
66     _METRICS = ("precision", "recall", "f1", "mean_iou")
67
68     class ExperimentError(ValueError):
69         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
70         score an unlabeled video with no pinned model, or a gate referencing an absent
71         feature)."""
72
73     # ----- Variant
74
75     @dataclass
76     class Variant:
77         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
78
79         A variant turns one video's persisted candidate windows + features into rally
80         intervals. Every knob defaults to the **control** behaviour (no gate, no learned
81         filter), so a variant that merely *carries* a new, disabled cue is byte-identical
82         to control ? the core no-regression guarantee.
83
84         Fields:
85         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
86           for the leaderboard and for selecting the served path (the existing
87           `segmenter_registry` + `IndexingConfig` do the actual selection in production).
88           New cues are recorded here default-OFF.

```

```

90     - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
91     ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92     ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93     windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md`` §3.1).
94     - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95     whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96     cue persists that feature).
97     - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98     ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99     required for ``compare_unlabeled`` on a learned variant.
100    - ``feature_names`` ? the persisted features the learned filter consumes. When set
101    WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102    (honest) ? the recipe, not a pinned artifact.
103    - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104    overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
105    """
106
107    name: str
108    segmenter: str = "wasb_hybrid"
109    config_overrides: Dict[str, Any] = field(default_factory=dict)
110    cue_flags: Dict[str, bool] = field(default_factory=dict)
111    min_crossings: int = 0
112    min_players: int = 0
113    classifier_model: Optional[str] = None
114    feature_names: Optional[List[str]] = None
115    threshold: float = 0.5
116    merge_gap: float = 1.0
117    # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118    # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119    # failure where a fixed 0.5 zeroed out a small-candidate video.
120    tune_threshold: bool = False          # pick the F1-best threshold on the train fold
121    calibrate: Optional[str] = None        # None | "platt" | "isotonic" ? calibrate
proba first
122
123    def is_learned(self) -> bool:
124        """True if this variant applies a learned classifier (pinned model or
recipe)."""
125        return self.classifier_model is not None or bool(self.feature_names)
126
127    def to_dict(self) -> dict:
128        return {
129            "name": self.name,
130            "segmenter": self.segmenter,
131            "config_overrides": self.config_overrides,
132            "cue_flags": self.cue_flags,
133            "min_crossings": self.min_crossings,
134            "min_players": self.min_players,
135            "classifier_model": self.classifier_model,
136            "feature_names": self.feature_names,
137            "threshold": self.threshold,
138            "merge_gap": self.merge_gap,
139            "tune_threshold": self.tune_threshold,
140            "calibrate": self.calibrate,
141        }
142
143    @classmethod
144    def from_dict(cls, d: dict) -> "Variant":
145        known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146        return cls(**{k: v for k, v in d.items() if k in known})
147
148    def spec_hash(self) -> str:
149        """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard

```

```

150         and makes a result reproducible. The pinned **control** variant always hashes
151         the
152         same, so a regression is always traceable to a recipe change, never to drift."""
153         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
154         return sha1(blob.encode()).hexdigest()[:12]
155
156     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157     #: filter. Always the comparison reference; "rollback" = make this the served variant.
158     CONTROL = Variant(name="control")
159
160
161     # ----- persisted candidates
162
163
164     @dataclass
165     class VideoCandidates:
166         """One video's persisted detection, ready for offline re-scoring.
167
168         ``intervals`` are the candidate windows; ``features`` is column-major
169         (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170         ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171         with ``load_video_candidates``, or directly in tests.
172         """
173
174         name: str
175         intervals: List[Interval]
176         features: Dict[str, List[float]] = field(default_factory=dict)
177         gts: Optional[List[Interval]] = None
178
179
180     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182         ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
183         col = vc.features.get(name)
184         n = len(vc.intervals)
185         if col is None:
186             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                            name, vc.name)
188             return [0.0] * n
189         if len(col) != n:
190             raise ExperimentError(
191                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192         return [float(x) for x in col]
193
194
195     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
196     List[List[float]]:
197         cols = [_feature_column(vc, name) for name in feature_names]
198         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
199
200     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202         players)."""
203         n = len(vc.intervals)
204         mask = [True] * n
205         if variant.min_crossings > 0:
206             col = _feature_column(vc, "net_crossings")
207             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208         if variant.min_players > 0:
209             col = _feature_column(vc, "players_in_court")
210             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211         return mask

```

```

212
213 def predict(variant: Variant, vc: VideoCandidates,
214             model: Optional[LogisticRegression] = None,
215             threshold: Optional[float] = None,
216             calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217     """Variant prediction: gate by persisted features, optionally filter by a learned
218     model, then merge. Pure and deterministic.
219
220     ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221     fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222     loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223     is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224     fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225     them.
226     """
227     mask = _gate_mask(variant, vc)
228     if variant.feature_names:
229         if model is None:
230             if variant.classifier_model is None:
231                 raise ExperimentError(
232                     f"variant {variant.name!r} is learned but has no model to predict "
233                     f"with (pass a fitted model or set classifier_model)")
234             model = LogisticRegression.load(variant.classifier_model)
235             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
236 variant.feature_names))]
237             if calibrator is not None:
238                 proba = [calibrator(p) for p in proba]
239                 thr = variant.threshold if threshold is None else threshold
240                 mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
241             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
242             return merge_overlaps(kept, gap_tol=variant.merge_gap)
243
244
245 def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
246                        train_videos: Sequence[VideoCandidates], iou: float
247                        ) -> "tuple[Optional[Callable[[float], float]],
248 Optional[float]]":
249     """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
250     (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
251     (use the variant default). Honest: never looks at the held-out video.
252     """
253     calibrator: Optional[Callable[[float], float]] = None
254     if variant.calibrate:
255         raw: List[float] = []
256         y: List[int] = []
257         for vc in train_videos:
258             raw.extend(float(p) for p in
259 model.predict_proba(_feature_matrix(vc, variant.feature_names or
260 [])))
261             y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
262         if variant.calibrate == "platt":
263             calibrator = fit_platt(raw, y)
264         elif variant.calibrate == "isotonic":
265             calibrator = fit_isotonic(raw, y)
266         else:
267             raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
268 'platt'/'isotonic')")
269     threshold: Optional[float] = None
270     if variant.tune_threshold:
271         best_t, best_f1 = variant.threshold, -1.0
272         for k in range(1, 20):
273             # grid 0.05..0.95 on the train
274             t = round(0.05 * k, 2)
275             f1s = [score(predict(variant, vc, model=model, threshold=t,
276 calibrator=calibrator),

```



```

270         vc.gts or [], iou).f1 for vc in train_videos]
271     mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272     if mean_f1 > best_f1:
273         best_f1, best_t = mean_f1, t
274     threshold = best_t
275     return calibrator, threshold
276
277
278     # ----- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
284         X: List[List[float]] = []
285         y: List[int] = []
286         for vc in videos:
287             if vc.gts is None:
288                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))
291         if not X:
292             raise ExperimentError("cannot train a learned variant on zero candidates")
293         return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299         mean aggregate.
300
301         Prediction policy:
302         - **static variant** (no learned filter): predict each video directly ? no
training,
303         so no leakage; per-video scoring is already honest.
304         - **learned, pinned model** (`classifier_model` set): score every video with the
305         SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
306         report "how the shipped artifact does here", not for the promotion decision.
307         - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
?
308         train on the OTHER videos, predict the held-out one. The number to trust.
309         - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
only).
310         """
311         for vc in videos:
312             if vc.gts is None:
313                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
314
315         per_video: List[Dict[str, Any]] = []
316         predictions: Dict[str, List[Interval]] = {}
317
318         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321         all_model = None
322         if recipe_lovo and not honest:
323             all_model = _train(variant, videos, iou)
324
325         for i, vc in enumerate(videos):
326             cal: Optional[Callable[[float], float]] = None
327             thr: Optional[float] = None

```

```

328         if recipe_lovo and honest:
329             others = [v for j, v in enumerate(videos) if j != i]
330             if not others:
331                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332             model: Optional[LogisticRegression] = _train(variant, others, iou)
333             if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334                 assert model is not None # _train never returns
None
335                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo: # honest=False ? one model over all
videos
337                 model = all_model
338             else: # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346             aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                          for m in _METRICS}
348         return {
349             "variant": variant.name,
350             "spec_hash": variant.spec_hash(),
351             "is_learned": variant.is_learned(),
352             "honest_lovo": recipe_lovo and honest,
353             "per_video": per_video,
354             "aggregate": aggregate,
355             "predictions": predictions,
356         }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])
375             ratios.append(_segm.segment_count_ratio(preds, gts))
376
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out

```

```

382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                        eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
387         EXPERIMENT_HARNESS §6).
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
389         order),
390         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
391         when
392         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
393         """
394         a_f1 = [p["f1"] for p in eval_a["per_video"]]
395         b_f1 = [p["f1"] for p in eval_b["per_video"]]
396         return {
397             "variant_a_ci": _ci_block(eval_a),
398             "variant_b_ci": _ci_block(eval_b),
399             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
400             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
401                             "variant_b": _segmentation_block(videos, eval_b)},
402         }
403
404     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
405                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
406     Dict[str, Any]:
407         """Offline labeled A/B (EXPERIMENT_HARNESS.md §5.1). Scores both variants on the
408         same
409         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
410
411         The deltas use the existing `metrics.score` (per video) +
412         `calibration.pct_improvement`;
413         learned variants are scored LOVO-honestly. The result is a self-contained record fit
414         for
415         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
416         it.
417
418         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
419         CIs, a
420         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
421         *alongside* the
422         existing bare-mean fields (additive: nothing existing changes). Set False for the
423         legacy
424         shape. This is the measurement-honesty wiring (ANALYZERS_AND_RECONCILERS.md
425         §13/C1+C4): a
426         bare LOVO mean at n?6 is not trustworthy on its own.
427         """
428         if len(videos) < 1:
429             raise ExperimentError("compare needs at least one labeled video")
430         eval_a = evaluate_variant(videos, variant_a, iou, honest)
431         eval_b = evaluate_variant(videos, variant_b, iou, honest)
432
433         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
434         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
435         m in _METRICS}
436
437         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
438         per_video_delta = [
439             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
440             for p in eval_b["per_video"]]
441
442         # One fingerprint over the whole label set ? detects "the deltas were measured
443         against

```

```

433     # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434     all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436     result: Dict[str, Any] = {
437         "iou": iou,
438         "label_set_hash": gt_hash(all_gts),
439         "num_videos": len(videos),
440         "variant_a": eval_a,
441         "variant_b": eval_b,
442         "delta": delta,
443         "delta_pct": delta_pct,
444         "per_video_delta": per_video_delta,
445     }
446     if honest_stats:
447         result["honest"] = honest_summary(videos, eval_a, eval_b)
448     return result
449
450
451     # ----- SxS on unlabeled video
452
453
454     def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455                           agree_iou: float = 0.5) -> Dict[str, Any]:
456         """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458         With no labels there is no lift to measure ? instead this surfaces where the two
459         variants **agree vs diverge**, which is exactly what a human reviews (and what two
460         highlight reels would show side by side):
461
462         - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463         - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464         - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
465           the human / a later golden label adjudicates).
466
467         Both variants must be predictable without labels: a learned variant therefore needs
468         a
469         pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470         ``metrics.match_intervals`` ? no new matching logic.
471         """
472         preds_a = predict(variant_a, video)
473         preds_b = predict(variant_b, video)
474         mr = match_intervals(preds_a, preds_b, agree_iou)
475
476         agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
477                   for m in mr.matches]
478         agree_ious = [m.iou for m in mr.matches]
479         a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
480         b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
481
482         def _dur(ivs: List[Interval]) -> float:
483             return sum(e - s for s, e in ivs)
484
485         return {
486             "video": video.name,
487             "agree_iou": agree_iou,
488             "variant_a": variant_a.name,
489             "variant_b": variant_b.name,
490             "num_a": len(preds_a),
491             "num_b": len(preds_b),
492             "num_agreed": len(agreed),
493             "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
494             "duration_a": _dur(preds_a),
495             "duration_b": _dur(preds_b),
496             "agreed": agreed,

```

```

496         "a_only": a_only,
497         "b_only": b_only,
498     }
499
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                      "spec_hash": result["variant_a"]["spec_hash"],
518                      "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                      "spec_hash": result["variant_b"]["spec_hash"],
521                      "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525     the
526     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
527     honest = result.get("honest") or {}
528     if honest.get("paired_delta_f1"):
529         row["honest_delta_f1"] = honest["paired_delta_f1"]
530     entries: List[Dict[str, Any]] = []
531     if os.path.isfile(path):
532         with open(path, encoding="utf-8") as f:
533             entries = json.load(f)
534     entries.append(row)
535     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
536     with open(path, "w", encoding="utf-8") as f:
537         json.dump(entries, f, indent=2)
538     logger.info("recorded experiment %s vs %s (?f1=%3f) -> %s",
539               row["variant_a"]["name"], row["variant_b"]["name"],
540               (result.get("delta") or {}).get("f1", 0.0), path)
541     return row
542
543 def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                          gts: Optional[List[Interval]] = None,
545                          feature_names: Sequence[str] = ()) -> VideoCandidates:
546     """Bridge persisted detection ? `VideoCandidates` via
547     `Database.query_candidate_segments`.
548     ``feature_names`` selects which persisted features to lift out of each row's
549     ``stats``
550     JSON (``"duration"`` is derived from the window bounds). This is the only DB-
551     touching
552     function; the comparison itself is pure and offline. Exercised owner-side on a real
553     DB,
554     but works against any `Database` (including an in-memory one in tests).
555     """
556     rows = db.query_candidate_segments(video_id, detector=detector)
557     intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
558     rows]

```

```

555     features: Dict[str, List[float]] = {}
556     for name in feature_names:
557         if name == "duration":
558             features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559         else:
560             features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561     return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562

```

6. assistant (2026-06-14T09:49:51.136Z)

Let me read the remaining files in parallel.

7. user (2026-06-14T09:49:52.095Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3     NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6     module only turns those signals into a handful of scale/translation-invariant
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12    - per_frame is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13    index (so it aligns directly with the shuttle trajectory's frame ? no clip re-
timing).
14    - A shuttle point has frame, visible, x, y (pixels) ? the WASB
trajectory shape.
15    - court_polygon is normalised 0-1 [(x, y), ...] or None (None ? no mask,
every
16    detection counts ? the player-count-only mode).
17    - net is (axis 'x'|'y', position 0-1 normalised) or None (None ? two-sided =
0.0).
18
19    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20    floats in [0, 1] except mean_player_motion (a non-negative normalised speed).
21    """
22    from __future__ import annotations
23
24    from typing import Dict, List, Optional, Sequence, Tuple
25
26    BBox = Tuple[float, float, float, float]
27    Net = Tuple[str, float]
28
29
30    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31        """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33        if polygon is None or len(polygon) < 3:
34            return False
35        x, y = point
36        inside = False
37        n = len(polygon)
38        j = n - 1
39        for i in range(n):

```

```

40         xi, yi = polygon[i]
41         xj, yj = polygon[j]
42         # Does the horizontal ray at y cross edge (i, j)?
43         if (yi > y) != (yj > y):
44             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45             if x < x_cross:
46                 inside = not inside
47         j = i
48     return inside
49
50
51 def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52     """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53     the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54     if frame_width <= 0 or frame_height <= 0:
55         return (0.0, 0.0)
56     x1, _y1, x2, y2 = box
57     return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60 def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61     if frame_width <= 0 or frame_height <= 0:
62         return (0.0, 0.0)
63     x1, y1, x2, y2 = box
64     return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67 def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                    court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69     """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70     True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71     point meaningless, so a masked check returns False rather than testing a bogus
72     (0,0)."""
73     if court_polygon is None:
74         return True
75     if frame_width <= 0 or frame_height <= 0:
76         return False
77     return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
78
79 def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                    frame_height: float,
81                    court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82     """Per-frame count of in-court persons (one entry per sampled frame)."""
83     return [
84         sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
85         court_polygon))
86         for _frame_idx, dets in sorted(per_frame.items())
87     ]
88
89 def _median(vals: Sequence[float]) -> float:
90     if not vals:
91         return 0.0
92     s = sorted(vals)
93     n = len(s)
94     return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97 def players_in_court(counts: Sequence[int]) -> float:
98     """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99     return _median(list(counts))

```

```

100
101
102 def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103     """Fraction of sampled frames with >= `min_players` in court (track stability vs
104 flicker)."""
105     if not counts:
106         return 0.0
107     return sum(1 for c in counts if c >= min_players) / len(counts)
108
109 def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                        frame_width: float, frame_height: float) -> float:
111     """Mean per-track centre displacement between consecutive sampled frames, with x
112 normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113 not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114     if frame_width <= 0 or len(per_frame) < 2:
115         return 0.0
116     frames = sorted(per_frame.keys())
117     steps: List[float] = []
118     for a, b in zip(frames, frames[1:]):
119         prev = {tid: box for tid, box in per_frame[a]}
120         for tid, box in per_frame[b]:
121             if tid in prev:
122                 cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                 cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                 steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125     if not steps:
126         return 0.0
127     return sum(steps) / len(steps)
128
129
130 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131                     frame_height: float, net: Optional[Net],
132                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
137     0.0.
138     """
139     if net is None or not per_frame:
140         return 0.0
141     axis, pos = net
142     both = 0
143     for _frame_idx, dets in per_frame.items():
144         near, far = False, False
145         for _tid, box in dets:
146             if not person_in_court(box, frame_width, frame_height, court_polygon):
147                 continue
148             fx, fy = _foot_norm(box, frame_width, frame_height)
149             coord = fx if axis == "x" else fy
150             if coord < pos:
151                 near = True
152             else:
153                 far = True
154         if near and far:
155             both += 1
156     return both / len(per_frame)
157
158 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
159 float,
160                    pad_frac: float) -> bool:
161     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
    width/height (the 'adjacent / in-hand' tolerance)"""

```



```

162     x1, y1, x2, y2 = box
163     px = pad_frac * frame_width
164     py = pad_frac * frame_height
165     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
169                                     frame_width: float, frame_height: float,
170                                     pad_frac: float = 0.03) -> float:
171         """Fraction of (visible shuttle frames that were also person-sampled) in which the
172         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
173         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
174         """
175         if not per_frame or frame_width <= 0 or frame_height <= 0:
176             return 0.0
177         aligned = 0
178         colocated = 0
179         for p in shuttle_points or []:
180             if not getattr(p, "visible", False):
181                 continue
182             dets = per_frame.get(int(p.frame))
183             if dets is None:         # shuttle frame wasn't person-sampled ? can't judge
184                 continue
185             aligned += 1
186             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
187                     for _tid, box in dets):
188                 colocated += 1
189         return colocated / aligned if aligned else 0.0
190
191
192     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194     PERSON_FEATURE_NAMES = [
195         "players_in_court", "two_sided_motion", "mean_player_motion",
196         "in_court_ratio", "shuttle_player_colocation",
197     ]
198
199
200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
201                                     frame_width: float, frame_height: float,
202                                     court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
203                                     net: Optional[Net] = None,
204                                     min_players: int = 2) -> Dict[str, float]:
205         """Aggregate the per-window person features from per-frame detections + the in-
window
206         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
207         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
208         return {
209             "players_in_court": players_in_court(counts),
210             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
211             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212             "in_court_ratio": in_court_ratio(counts, min_players),
213             "shuttle_player_colocation": shuttle_player_colocation(
214                 per_frame, shuttle_points, frame_width, frame_height),
215         }
216

```

8. user (2026-06-14T09:49:52.568Z)

```

1         """Pure per-window audio-cue features for rally fusion (fusion P4).

```

```

2
3     NO I/O ? the audio hit detection lives in `backend/pipeline/audio/hit_detector.py`
4     (`detect_hits_in_video` -> `List[HitEvent]`); this only turns those hit events into a
few
5     per-window features the learned scorer can weight. Mirrors `fusion_features.py`
discipline.
6
7     Honest expectation (see the `audio-e1-findings` memory): the classical-onset audio cue
had
8     only ~2.2x discrimination and was NOT adopted standalone ? so this is "add the signal as
a
9     column and let the golden-set harness decide", default-OFF, not "adopt audio".
10    ""
11    from __future__ import annotations
12
13    from typing import Dict, List
14
15    AUDIO_FEATURE_NAMES = ["audio_hit_count", "audio_hit_rate", "audio_hit_strength_mean"]
16
17
18    def compute_audio_features(hits: List, window_start: float, window_end: float) ->
Dict[str, float]:
19        """Audio features for one window from a video's HitEvents (each has ``.t``,
``.strength``).
20
21        A real rally is a sustained volley of racket/shuttle contacts; dead time / a single
22        service feed has few. Returns hit count, hits/second, and mean strength ? all 0.0 on
an
23        empty window. ``[start, end)`` half-open so a hit on a boundary belongs to one
window only.
24        ""
25        dur = max(1e-6, window_end - window_start)
26        in_win = [h for h in (hits or []) if window_start <= h.t < window_end]
27        if not in_win:
28            return {k: 0.0 for k in AUDIO_FEATURE_NAMES}
29        n = len(in_win)
30        return {
31            "audio_hit_count": float(n),
32            "audio_hit_rate": n / dur,
33            "audio_hit_strength_mean": sum(h.strength for h in in_win) / n,
34        }
35

```

9. user (2026-06-14T09:49:53.047Z)

```

1     """Per-user promotion gate ? the safety rail for flag-gated personalization.
2
3     The owner adopted personalization as a **flag-gated** feature on a **generalization-
first**
4     baseline: a cue/variant B may be flipped ON *for one user* only when that user's own
5     held-out evidence honestly clears the same small-N bar the global eval uses ? otherwise
we
6     fall back to the shipped baseline (variant A). This module is the decision logic for
that
7     flip; it is pure (no I/O) and **reuses** ``backend.eval.eval_stats`` for every statistic
8     (``paired_delta_ci`` + ``paired_sign_test``) so the math is never reinvented.
9
10    The counter-case this guards against (and its mitigations, implemented here):
11
12    1. **Minimum-N floor.** Personalizing off 1-2 of a user's videos is the small-N-lies
regime
13        (see ``eval_stats`` C1). Refuse to promote below ``min_n`` videos ? point estimates
from
14        too few held-out videos are noise, not signal.
15    2. **Honest lift, not a single mean.** Promote only when the paired ?F1 95% bootstrap CI

```

```

16         **excludes 0** *and* the paired sign test agrees (majority of videos win, p <=
threshold).
17         This is the global promotion bar applied per user.
18     3. **Structural guards are never disabled on "no lift".** A *structural* guard (e.g.
Gate-A
19         net-crossings ? the held-shuttle false-positive killer) earns its keep through
failure
20         modes a given user's corpus may not contain yet. Measuring "no per-user lift" on a
corpus
21         that simply lacks those failure cases must NEVER turn the guard OFF. For
``structural=True``
22         the decision is therefore **always "keep ON"**, independent of the measured delta.
23
24         Below the floor / insufficient evidence ? fall back to the shipped baseline (the
conservative
25         default), never an unsafe promotion. All additive, pure, deterministic (the bootstrap
seed is
26         threaded through). Nothing here changes existing eval behaviour; callers opt in.
27         """
28         from __future__ import annotations
29
30         from dataclasses import dataclass, field
31         from typing import Any, Dict, Sequence
32
33         from backend.eval.eval_stats import paired_delta_ci, paired_sign_test
34
35         __all__ = ["PromotionDecision", "should_promote_for_user"]
36
37
38         @dataclass
39         class PromotionDecision:
40             """Structured result of a per-user promotion decision.
41
42             - ``promote`` ? flip variant B ON for this user (only ever ``True`` for a non-
structural
43             cue that cleared the full bar).
44             - ``keep_on`` ? the cue/guard should be ON for this user. For a structural guard
this is
45             always ``True`` (it is never disabled on "no lift"); for a non-structural cue it
equals
46             ``promote``.
47             - ``reason`` ? short human-readable explanation of the decision.
48             - the remaining fields surface the evidence (n, mean delta, CI bounds and the two
tests)
49             so a caller can log/audit exactly why a user was or wasn't personalized.
50             """
51
52             promote: bool
53             keep_on: bool
54             reason: str
55             n: int
56             mean_delta: float
57             ci_low: float
58             ci_high: float
59             ci_excludes_zero: bool
60             sign_test: Dict[str, float] = field(default_factory=dict)
61             floor_met: bool = False
62             structural_protected: bool = False
63
64             def as_dict(self) -> Dict[str, Any]:
65                 """Plain-dict view (for JSON logging / telemetry)."""
66                 return {
67                     "promote": self.promote,
68                     "keep_on": self.keep_on,
69                     "reason": self.reason,

```

```

70         "n": self.n,
71         "mean_delta": self.mean_delta,
72         "ci_low": self.ci_low,
73         "ci_high": self.ci_high,
74         "ci_excludes_zero": self.ci_excludes_zero,
75         "sign_test": self.sign_test,
76         "floor_met": self.floor_met,
77         "structural_protected": self.structural_protected,
78     }
79
80
81     def should_promote_for_user(
82         a_scores: Sequence[float],
83         b_scores: Sequence[float],
84         *,
85         structural: bool = False,
86         min_n: int = 5,
87         p_threshold: float = 0.05,
88         confidence: float = 0.95,
89         n_boot: int = 2000,
90         seed: int = 0,
91         eps: float = 0.0,
92     ) -> PromotionDecision:
93         """Decide whether to flip variant B ON for ONE user from that user's held-out
94         scores.
95         ``a_scores`` / ``b_scores`` are the user's per-video held-out F1 (or any score)
96         for
97         variant A (control / shipped baseline) and variant B (with the cue), aligned by
98         video
99         (same order). The decision rules (faithful to the module docstring):
100
101         1. ``structural=True`` ? the guard is never disabled on "no lift": return
102           ``keep_on=True, promote=False`` regardless of the measured per-user delta. Its
103           value is
104           in failure modes the user's corpus may not yet contain.
105
106         2. Otherwise promote iff ALL hold ? the minimum-N floor (``n >= min_n``), the
107           paired
108           ?F1 bootstrap CI excludes 0, and the paired sign test agrees (majority of
109           videos
110           win, ``p_value <= p_threshold``). Any miss ? fall back to the shipped baseline.
111
112         Statistics come straight from ``eval_stats`` (``paired_delta_ci`` /
113         ``paired_sign_test``);
114         deterministic given ``seed``. Returns a :class:`PromotionDecision`.
115         """
116         n = min(len(a_scores), len(b_scores))
117         floor_met = n >= min_n
118
119         # --- Evidence (reuse eval_stats; never reinvent the statistics)
120         -----
121         if n == 0:
122             # No paired videos at all ? nothing to test. Surface an empty-but-honest result.
123             sign_test: Dict[str, float] = paired_sign_test([], eps)
124             mean_delta, ci_low, ci_high, ci_excludes_zero = 0.0, 0.0, 0.0, False
125         else:
126             delta = paired_delta_ci(
127                 a_scores, b_scores,
128                 confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
129             )
130             mean_delta = float(delta["mean_delta"])
131             ci_low = float(delta["ci_low"])
132             ci_high = float(delta["ci_high"])
133             ci_excludes_zero = bool(delta["ci_excludes_zero"])
134             sign_test = delta["sign_test"]

```

```

127
128     sign_p = float(sign_test.get("p_value", 1.0))
129     sign_wins = float(sign_test.get("wins", 0.0))
130     sign_losses = float(sign_test.get("losses", 0.0))
131     # The sign test "agrees" with a promotion only if it is significant AND the majority
132     # leans the *winning* way (more B-wins than B-losses) ? a significant net *loss*
must
133     # never satisfy the promotion bar.
134     sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
135
136     # --- Rule 1: structural guards are never disabled on "no lift"
-----
137     if structural:
138         return PromotionDecision(
139             promote=False,          # nothing to "promote" ? it ships ON by design
140             keep_on=True,          # ALWAYS kept ON, regardless of measured per-user
lift
141             reason=(
142                 "structural guard kept ON regardless of measured per-user lift "
143                 "(value is in failure modes this user's corpus may not yet contain)"
144             ),
145             n=n,
146             mean_delta=mean_delta,
147             ci_low=ci_low,
148             ci_high=ci_high,
149             ci_excludes_zero=ci_excludes_zero,
150             sign_test=sign_test,
151             floor_met=floor_met,
152             structural_protected=True,
153         )
154
155     # --- Rule 2: minimum-N floor (refuse before even looking at lift)
-----
156     if not floor_met:
157         return PromotionDecision(
158             promote=False,
159             keep_on=False,          # fall back to the shipped baseline (variant A)
160             reason=(
161                 f"below minimum-N floor (n={n} < min_n={min_n}); "
162                 "falling back to shipped baseline"
163             ),
164             n=n,
165             mean_delta=mean_delta,
166             ci_low=ci_low,
167             ci_high=ci_high,
168             ci_excludes_zero=ci_excludes_zero,
169             sign_test=sign_test,
170             floor_met=False,
171             structural_protected=False,
172         )
173
174     # --- Rule 2 (cont.): honest lift ? CI excludes 0 AND sign test agrees
-----
175     if ci_excludes_zero and sign_agrees:
176         promote = True
177         reason = (
178             f"promoted: n={n} >= {min_n}, ?F1 95% CI "
179             f"[{ci_low:.4f}, {ci_high:.4f}] excludes 0, "
180             f"sign test {int(sign_wins)}-{int(sign_losses)} (p={sign_p:.4f} <=
{p_threshold})"
181         )
182     else:
183         promote = False
184         misses = []
185         if not ci_excludes_zero:

```

```

186         misses.append(f"?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] spans 0")
187     if not sign_agrees:
188         misses.append(
189             f"sign test does not agree ({int(sign_wins)}-{int(sign_losses)}), "
190             f"p={sign_p:.4f}")
191     )
192     reason = (
193         "insufficient evidence (" + "; ".join(misses) + "); "
194         "falling back to shipped baseline"
195     )
196
197     return PromotionDecision(
198         promote=promote,
199         keep_on=promote,          # non-structural: keep_on tracks promote exactly
200         reason=reason,
201         n=n,
202         mean_delta=mean_delta,
203         ci_low=ci_low,
204         ci_high=ci_high,
205         ci_excludes_zero=ci_excludes_zero,
206         sign_test=sign_test,
207         floor_met=True,
208         structural_protected=False,
209     )
210

```

10. assistant (2026-06-14T09:49:55.965Z)

Now let me read the three docs that frame the signal-fusion model and the quality log this feeds.

11. user (2026-06-14T09:49:57.167Z)

```

1     # Experiment harness ? A/B + shadow eval for rally-detection quality (no-regression by
design)
2
3     **Status:** *P1 IMPLEMENTED* (`backend/eval/experiment.py` +
`tests/test_experiment.py`, shipped
4     2026-06-13) ? the `Variant` layer, `compare()` (labeled A/B, LOVO-honest),
`compare_unlabeled()`
5     (SxS on NEW unlabeled footage), and the leaderboard. **P2?P4 remain owner-gated.**
Phases below are
6     sequenced but each is a separate PR.
7
8     > **Companion docs:** the cues this experiments *on* are in
9     > [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md); the scoring mechanics are
10    > [EVALUATION.md](EVALUATION.md); the no-regression gate + golden labels +
`baseline.json` come from
11    > [GOLDEN_SET_VENDORIZING.md](GOLDEN_SET_VENDORIZING.md); the modular-cue architecture is
**ADR-007**
12    > ([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md)); the distilled
classifier is
13    > **ADR-004**. The 2-layer mental model is
[HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md).
14
15    ---
16
17    ## 1. Context ? why this, why now
18
19    PR #112 (merged) added the multi-signal fusion design ? shuttle + person + audio cues,
all **default
20    OFF**. The owner's directive for *how* we evolve quality from here:
21
22    > "Evolve the codebase so we can keep finding new ways to improve quality and
experiment with them ?

```

```

23     > or add them as a signal ? without suffering any regressions if the idea doesn't work
out. Rolling
24     > back the code would be too dangerous.**
25
26     So **experimentation and deployment must be decoupled.** The contract:
27
28     1. Merge experimental code freely ? it's **gated OFF**, so it cannot change production
behavior.
29     2. Test it **offline against golden labels** (most experiments never touch the served
path at all).
30     3. Promote a variant to default **only on measured lift**, through a CI eval gate.
31     4. **"Rollback" = flip a config flag, never `git revert`.** The proven path is a pinned
variant that is
32         always present.
33
34     **The pleasant surprise (Explore audit, 2026-06-13):** ~80% of this harness already
exists and is
35     simply not composed *as a system*. The missing piece is a thin **variant/experiment
layer + the
36     discipline**, not a platform. §4 is the exact code surface so the next session does not
re-discover it.
37
38     ---
39
40     ## 2. The thesis ? separate expensive DETECTION from cheap DECISION
41
42     The expensive, GPU/WSL-bound, risky part is **detection**: "what signals exist per
frame" (shuttle
43     WASB trajectory, person tracks/boxes, audio hits). The cheap, swappable, pure-Python
part is the
44     **decision**: "given those signals, where are the rallies" (windowing + gates + the
classifier).
45
46     > **Run detection once per video, persist all per-cue signals + per-candidate features,
then re-score
47     > any number of variants OFFLINE ? deterministically, with no GPU and zero production
risk.**
48
49     This is not aspirational ? `distill_local.py` and `calibrate_local.py` already do
exactly this on
50     stored WASB trajectories. The candidate-segments table (`stats` JSON) is the persistence
substrate.
51     ? Most experiments are pure re-scoring of stored data; they never enter the served
pipeline.
52
53     ---
54
55     ## 3. What already exists (the audit ? do NOT rebuild)
56
57     Exact public surface, with `file:line`. **This table is the anti-duplicate-work
artifact** ? start here.
58
59     ### 3.1 Scoring (variant-agnostic ? grades any `List[Interval]`)
60     `backend/eval/metrics.py`
61     - `interval_iou(a, b) -> float` (:15) ? temporal IoU of two `(start,end)` intervals.
62     - `match_intervals(preds, gts, iou_threshold=0.5) -> MatchResult` (:48) ? greedy 1-to-1
IoU matching.
63     - `score(preds, gts, iou_threshold=0.5, match_result=None) -> Score` (:104) ? P/R/F1,
mean IoU, boundary errors; `Score.as_dict()`.
64     - `pr_curve(preds, gts, thresholds=None) -> List[Score]` (:138).
65
66     ### 3.2 A/B + cross-validation primitives (the comparison engine already exists)
67     `backend/eval/calibration.py`
68     - `improvement_report(predict_fn, baseline_config, tuned_config, gts, iou_threshold=0.5)
-> dict` (:148) ? **before/after metrics + %?. This IS the A/B delta.**

```

```

69     - `leave_one_video_out(videos, configs, metric="f1", iou_threshold=0.5) -> dict` (:113)
? **LOVO: pick on other videos, score on held-out video (honest cross-video).**
70     - `cross_validate(predict_fn, configs, gts, k=5, metric="recall", iou_threshold=0.5) ->
dict` (:72) ? within-video k-fold overfit guard.
71     - `select_best(predict_fn, configs, gts, metric="f1", iou_threshold=0.5) -> (Config,
float)` (:61).
72     - `evaluate(preds, gts, iou_threshold=0.5) -> Score` (:41); `kfold_indices(n, k)` (:46).
73     - LOVO video dict shape: `{"name": str, "predict_fn": PredictFn, "gts": [Interval]}`.
74
75     ### 3.3 No-regression gate + baselines (the promotion gate already exists)
76     `backend/eval/regression.py`
77     - `regress(db, video_id, ground_truth, stage="final", iou_threshold=0.5, detector=None,
tolerance=DEFAULT_TOLERANCE, baseline_path=DEFAULT_BASELINE_PATH) -> RegressionResult` (:104) ?
score stored preds vs GT, flag if P **or** R drops > tolerance.
78     - `regress_with_provider(...)` (:160) ? same, fetching GT via the annotation provider.
79     - `save_baseline(video_id, stage, precision, recall, f1, ..., gt_fingerprint=None)`
(:80) . `load_baselines(path) -> dict` (:68).
80     - `RegressionResult` (:43): `regressed`, `reasons`, `gt_hash`, baseline P/R,
`tolerance`, `has_baseline`; `as_dict()`.
81     - Default baseline file: `eval_baselines/regression_baseline.json` (:33).
82
83     `run_regression.py` ? CI-gatable CLI, `main(argv)` (:52). **Exit codes: 0 ok / 1
regression / 2 no-DB / 3 no-baseline.** Flags: `--video-id --tier --annotations
--stage{candidates,final} --detector --iou --tolerance --baseline --update-baseline --allow-
missing-baseline --json`.
84
85     ### 3.4 Pinnable model artifact
86     `backend/eval/classifier.py` ? `LogisticRegression` (pure numpy, L2):
`fit(X,y,feature_names=,class_weight="balanced")` (:40), `predict_proba` (:75),
`predict(threshold=0.5)` (:81), `to_dict`/`from_dict` (:86/:97), `save`/`load` (JSON)
(:107/:111). **A trained model = one JSON file ? a pinnable, hashable variant artifact.**
87
88     ### 3.5 Offline re-scoring substrate (persist once, re-score forever)
89     `backend/infrastructure/database.py`
90     - `add_candidate_segment(video_id, detector, start_time, end_time, chunk_index=None,
confidence=None, stats=None, detection_params=None) -> Optional[int]` (:240) ?
`stats`/`detection_params` stored as JSON; `candidate_segments` table cols incl. `stats`,
`detection_params`, `human_label`, `confirmed_segment_id` (schema :97).
91     - `query_candidate_segments(video_id, detector=None, only_unlabeled=False) ->
List[dict]` (:277) ? returns rows with deserialized `stats`.
92
93     ### 3.6 Label assignment + feature glue
94     `backend/eval/training.py`
95     - `label_candidates(candidate_intervals, gt_intervals, iou_threshold=0.5) -> List[int]`
(:50) ? y-labels by IoU match to golden (same matcher as the evaluator).
96     - `build_training_set(candidate_rows, gt_intervals, iou_threshold=0.5,
feature_fn=extract_yolo_features) -> (X, y, intervals)` (:64) ? pluggable `feature_fn`.
97
98     ### 3.7 Existing LOVO drivers to mirror (not duplicate)
99     - `backend/eval/distill_local.py` ? `run(specs, iou=0.5, threshold=0.5, model_out=None)`
(:98), CLI `python -m backend.eval.distill_local --manifest videos.json --model-out ...`;
`FEATURE_NAMES = [duration, peak_velocity, mean_velocity, active_ratio, net_crossings]` (:40);
already does LOVO over Gemini silver labels.
100     - `backend/eval/calibrate_local.py` ? `run(specs, iou=0.5, metric="f1")` (:77);
`LocalConfig = (velocity_thresh, merge_gap, min_window_duration, min_crossings)`; grid + LOVO.
101     - `backend/eval/calibrate_wash.py` ? `run(...)` , `load_golden_gts()`;
`backend/eval/gt_loader.py::load_gt_intervals(csv_path, *, strict=True)` is THE canonical golden
reader.
102
103     ### 3.8 Registries + config knobs (variant selection)
104     - `backend/pipeline/segmenters/base.py` ? `SegmenterRegistry` (:35), singleton
`segmenter_registry` (:63); registered: `motion`, `gemini`, `yolo_hybrid`, `tracknet_hybrid`,
`wash_hybrid`.
105     - `backend/annotations/base.py` ? `AnnotationProviderRegistry`, singleton
`annotation_registry`; providers `file`, `auto`.

```



```

106     - `backend/config/models.py::IndexingConfig` ? `default_segmenter` (:65, default
`"yolo_hybrid"`), `detector_model` (:72), `detector_score_threshold` (:73),
`yolo_velocity_threshold` (:74), `yolo_frame_skip` (:75), `min_segment_duration` (:68),
`dead_time_merge_gap` (:69), `motion_threshold` (:67). **No `min_crossings` knob yet** (see
fusion P2).
107
108     ### 3.9 Confirmed ABSENT (no duplication risk)
109     Explore found **no** existing experiment / variant / shadow / A-B *machinery* ? only a
stray "A/B test"
110     aspiration comment in `backend/pipeline/detectors/base.py` and an "experiment E1" note
in
111     `AUDIO_RALLY_DETECTION_PLAN.md`. The `regress()` + baseline path is the canonical
comparison mechanism.
112
113     ---
114
115     ## 4. The variant abstraction (the one thin thing to add)
116
117     A **`Variant` = a declarative recipe**, not a code branch:
118
119     ```
120     Variant = (segmenter_name, IndexingConfig overrides, enabled cues + per-cue params,
classifier model path/hash)
121     ```
122
123     JSON-serializable; selected via the existing `segmenter_registry` +
`IndexingConfig.default_segmenter`.
124     **Control** = a *pinned, frozen* variant (recipe + model hash) that is always registered
and is the
125     default. Adding a new cue = a new Variant differing by one flag ? the control recipe is
untouched.
126
127     ---
128
129     ## 5. Three experiment modes
130
131     ### 5.1 Offline A/B (primary, label-driven, zero production risk)
132     `compare(videos, variant_A, variant_B)` ? a thin driver composing **existing**
functions:
133     `query_candidate_segments()` (load persisted features) ? re-score each variant ?
`calibration.improvement_report()`
134     + `calibration.leave_one_video_out()` + `metrics.score()`, graded vs golden
(`gt_loader.load_gt_intervals`).
135     Reports per-video **and** LOVO-honest aggregate IoU/P/R/F1 deltas. **This is the A/B
test, and it needs
136     almost no new scoring code ** ? only the glue.
137
138     **Honest reporting (default ON, additive):** `compare(..., honest_stats=True)` also
attaches a **`honest`**
139     block ? per-metric bootstrap **CIs**, a paired **?F1 CI + exact sign test** (C1), and
**F1@k +
140     segment-count ratio ** (C4) ? *alongside* the bare-mean fields (existing output
unchanged; `honest_stats=False`
141     restores the legacy shape). A bare LOVO mean at n?6 is not promotable on its own; the
lift is real when the
142     ?F1 CI clears 0 **and** the sign test agrees. Wiring of the course corrections in
143     [ANALYZERS_AND_RECONCILERS.md](ANALYZERS_AND_RECONCILERS.md) §13/C1+C4 (helpers:
`backend/eval/eval_stats.py`,
144     `segmentation_metrics.py`); `record_experiment` carries the honest ?F1 into the
leaderboard.
145
146     ### 5.2 Shadow (for cues that touch detection, e.g. the person detector)
147     The new cue runs and **persists its signal** into the candidate
`stats`/`detection_params`, but the
148     *served* decision stays control. Offline you then ask "what would variant B have

```

```

scored?" via §5.1 ?
149     B never affects output until it wins. This is how a detection-touching change (not just
a re-scoring
150     change) earns its way in without risk.
151
152     ### 5.3 Promotion gate (no-regression, CI-enforced)
153     `regression.regress()` + `baseline.json` + `run_regression.py` exit codes. A variant
becomes the new
154     default **only if** it doesn't regress the golden set beyond `tolerance` (exit 1
blocks). Control is
155     pinned ? **rollback = flip `default_segmenter`/variant config, never a code revert.**
156
157     ---
158
159     ## 6. No-regression guarantees (the heart of the owner's ask)
160
161     - **New cues/signals default OFF** ? per-cue enable flag in `IndexingConfig`; merged
code can't change
162     behavior until explicitly enabled.
163     - **Control variant pinned** ? frozen recipe + classifier model hash, always registered
& default.
164     - **Promotion only via the eval gate** ? `run_regression.py` in CI (exit 1 blocks); no
silent default change.
165     - **Experiments are records** ? variant-spec hash ? per-video + LOVO scores + label-set
hash + timestamp,
166     persisted to a small JSON **experiment leaderboard** (generalizes `baseline.json`).
Answers "which
167     signals ever helped, on which video slices," and makes every result reproducible.
168     - **Decoupled events** ? "merge an experiment" and "change the default" are separate,
independently
169     revertible actions.
170
171     ---
172
173     ## 7. What to build later (gap list ? thin, additive, owner-gated)
174
175     1. ~**`backend/eval/experiment.py`** ? a `Variant` dataclass + `compare(videos, a, b)`
composing the §3
176     functions (no new scoring math).~ **DONE (2026-06-13).** Shipped `Variant` (+ pinned
`CONTROL`),
177     `compare` (labeled A/B with LOVO-honest learned-variant training, mirroring
`distill_local`),
178     `compare_unlabeled` (the **SxS-on-new-videos** mode ? agreed / A-only / B-only via
`match_intervals`,
179     no GT), `record_experiment` (? `eval_baselines/experiment_leaderboard.json`), and
`load_video_candidates`
180     (the `query_candidate_segments` bridge). JSON variant recipes first as recommended; a
`VariantRegistry`
181     only if the count grows. 19 tests incl. the no-regression invariant.
182     2. **Persist per-cue features** into candidate `stats` (person: `players_in_court`,
`two_sided_motion`,
183     `in_court_ratio`, `shuttle_player_colocation`; shuttle-state: `net_crossings`,
`direction_reversals`;
184     audio: hit times) so any future variant re-scores offline ? ties to
`MULTI_SIGNAL_FUSION_PLAN.md` §3.2/§4.
185     3. **Per-cue enable flags** in `IndexingConfig` (default `False`); `default_segmenter`
already exists.
186     Add the `min_crossings` knob (fusion P2) here too.
187     4. **CI promotion lane** using `run_regression.py` against the golden `baseline.json`;
write the
188     experiment-leaderboard JSON.
189
190     ---
191
192     ## 8. Phasing (owner-gated; ONE PR per slice, off `master`)

```

```

193
194     - **P0 ? this doc**, indexed in `docs/README.md`, cross-linked from
`MULTI_SIGNAL_FUSION_PLAN.md` and
195     `GOLDEN_SET_VENDORING.md`. *(this deliverable; docs-only, no code.)*
196     - **P1 ? `experiment.py` `compare()`** ? **DONE (2026-06-13):** thin wrapper over
`calibration` +
197     `metrics` + `query_candidate_segments`; pure-logic, fully tested in this dev container
(`compare`,
198     `compare_unlabeled`, `record_experiment`, the no-regression invariant).
`compare_unlabeled` adds the
199     SxS read on **new unlabeled footage** the owner asked for (run control + candidate on
a fresh video,
200     diff where they agree/diverge ? two reels via the existing `VideoCompiler` is the GPU-
side follow-on).
201     - **P2 ? per-cue enable flags + feature persistence** into candidate `stats` (default
OFF) ? couples
202     with fusion P1/P2.
203     - **P3 ? CI promotion gate**: `run_regression.py` lane on the golden `baseline.json`;
experiment-leaderboard JSON.
204
205     ---
206
207     ## 9. Scope guardrail
208
209     **Single-tenant / local-first** (per `PLATFORM_ARCHITECTURE.md`): a JSON experiment
registry + the
210     existing harness ? explicitly **NOT** MLflow / W&B / a service. Keep it the size of
`baseline.json`.
211
212     ---
213
214     ## 10. Verification
215
216     - **Offline, runnable in this dev container (no GPU/torch needed for the glue):** unit
test that
217     `compare()` is deterministic on synthetic candidate rows; a **no-regression invariant
test** asserting
218     "all new-cue flags OFF ? predictions byte-identical to control"; a CI-gate test
asserting
219     `run_regression.py` exit codes (0 ok / 1 regress / 3 no-baseline).
220     - **Machine-side (owner, GPU/WSL):** persist candidates once on a real clip, run
`compare(control,
221     fusion)` LOVO, confirm the reported lift matches a manual `metrics.score()` spot-
check.
222     - **Docs (P0):** links resolve; indexed in `docs/README.md`; cross-linked from
223     `MULTI_SIGNAL_FUSION_PLAN.md` and `GOLDEN_SET_VENDORING.md`.
224     </content>
225

```

12. user (2026-06-14T09:49:57.486Z)

```

1     # Rally-detection quality iterations ? living log
2
3     **Purpose.** A running, reverse-chronological log of how we evolve rally-detection
quality: each
4     iteration's hypothesis, what shipped, the measured (or expected) effect, and the lesson.
This is the
5     **live** continuation of the historical trail in
6     [`archives/quality-iterations/`](archives/quality-iterations/README.md) ? per the
archive policy
7     (`CLAUDE.md`), **terminal-state** (DONE/REJECTED) work gets a dated subdir there; this
file stays at
8     `docs/` top level and keeps growing, then its entries graduate to the archive when they
conclude.
9

```

```

10     **Methodology (non-negotiable).** Every quality change rides the experiment harness
11     ([EXPERIMENT_HARNESS.md])(EXPERIMENT_HARNESS.md): new cues land **flag-gated, default
OFF**;; they're
12     graded **offline against golden labels** (per-video + LOVO-honest); promotion to default
happens **only
13     on measured lift** through the `run_regression.py` gate; **rollback = flip a config
flag, never
14     `git revert`.** Cues are designed in
[MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md); scoring
15     in [EVALUATION.md](EVALUATION.md); the 2-layer model in
16     [HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md).
17
18     **Golden corpus = the 6 labelled matches** in `output/human_lovo_manifest.json` (owner-
confirmed
19     2026-06-13: use all 6 ? `adarsh_avi_singles, kushagra_singles, largetest_doubles,
gbaaddy,
20     testlarge_short, mahadevpura_singles`). The model trained on these is what we run
alongside the existing
21     detector on NEW footage for the side-by-side review.
22
23     ---
24
25     ## Baselines that drive strategy (IoU 0.5 vs human golden labels)
26
27     | Path | Mahadevpura (clean-ish) | Kushagra (multi-court) |
28     |---|---|---|
29     | Heuristic (velocity windowing) | 0.657 | 0.157 |
30     | Gen-0 owned head (LOVO, n=6) | 0.744 | 0.561 |
31     | Two-stage WASB+Gemini refine | 0.83 | ? |
32     | Gemini wrapper (`gemini-flash-latest`) | 0.93 | 0.66 |
33
34     **Reading:** clean footage is commoditized (serve via Gemini); **multi-court drops the
commodity wrapper
35     to 0.66 ? that is the owned model's ship target.** Baselines tracked in
`eval_baselines/`. The owner's
36     observed false positive ? a player **holding/flicking** the shuttle between points
counted as a rally ?
37     is the precision bug the fusion gates target.
38
39     ---
40
41     ## Current track (live) ? newest first
42
43     ### Fusion P3 + P4 MEASURED on the n=6 golden corpus ? person = no lift, audio =
promising (not promotable) *(2026-06-14)*
44     The GPU person-feature extraction finished all 6 golden videos (`fusion_golden.py` ?
`output/*_golden_features.json`, the
45     replayable substrate); the CPU LOVO drivers then measured the lift with **honest stats
(C1 ? bootstrap 95% CI + paired sign
46     test; never a bare mean at n=6)**:
47
48     | Variant (honest LOVO, IoU ? 0.5) | F1 | ?F1 vs prior | 95% CI | sign test | CI clears
0 |
49     |---|---|---|---|---|---|
50     | shuttle-only | 0.307 | ? | ? | ? | ? |
51     | + person (P3) | 0.286 | **?0.021** (?6.7%) | [?0.165, +0.162] | 2W/4L (p=0.688) |
**No** |
52     | + person + audio (P4 incr.) | 0.366 | **+0.079** (+27.7%) | [?0.054, +0.203] | 4W/2L
(p=0.688) | **No** |
53
54     **Read honestly:** neither cue clears the promotion bar (both CIs span 0, both sign
tests n.s.) at n=6 ? **both stay
55     default-OFF** (promote-only-on-lift). **Person** adds noise (no court polygons ?
`players_in_court` counts spectators);
56     per-video it's a wash (testlarge +0.40, gbaaddy ?0.22). **Audio** is the more promising

```

lead ? +0.079 / +27.7%, wins 4/6

57 (largetest +0.27, mahadevpura +0.27), and shuttle+person+audio (0.366) lands ****above****
shuttle-only (0.307) ? but n=6 can't

58 confirm it. ****Next:**** more matches (raise N until the sign test can resolve ~+0.08), and
measure audio on shuttle-only

59 ***directly*** (without the person handicap). Recorded in
`eval\_baselines/experiment\_leaderboard.json`. ****Negative/uncertain**

60 results are kept on purpose ? this is the honest ablation backbone for the paper aim.******
61

62 **###** Acting on the first A/B: Gate-A PROMOTED + learned variant FIXED + GX010125
corroboration *(2026-06-14)*

63 ****Gate-A (net-crossings ? 2) ? PROMOTED (first cue promoted via the harness).**** The
measured win

64 (?F1 +0.074, ****6/6 golden matches, sign-test p=0.031****, CI [+0.042,+0.104]; over-seg
1.68?1.01 ? the

65 [first-A/B archive](archives/quality-iterations/2026-06-first-ab-experiment/README.md))
clears the honest

66 bar. ****Recommended config:**** `indexing.fusion.min\_crossings = 2` on the fusion path (the
eval-only Gate A

67 folded into production, MULTI_SIGNAL_FUSION_PLAN §3.1). ? The offline win is on the
recall-oriented ***proposal***

68 candidate set; flipping the ****live served default**** stays owner-gated until a real
fusion-path run confirms it

69 (the production fusion segmenter is P2/unverified on real video) ? so this records the
promotion + rationale,

70 not a silent default flip. ****Corroborated on real footage:**** offline re-score of
GX010125_1080p's ***cached***

71 trajectory (no GPU, isolated `output/GX010125\_run/`) ? 45 candidates ? CONTROL 45
rallies vs ****Gate-A 27****;

72 Gate-A dropped ****18 low-crossing windows (~1.7 min)**** of likely held-shuttle/feed FPS
(no golden labels for

73 this footage ? divergence, not F1).

74

75 ****Learned variant FIXED ? [#130](https://github.com/avidullu/badminton-highlight-
indexer/pull/130).**** The

76 first A/B's learned logistic lost to CONTROL and zeroed `testlarge` ? a fixed-0.5-cutoff
artifact. Added

77 additive/default-OFF `Variant.tune\_threshold` (operating point chosen on the ****train****
fold) + `calibrate`

78 (Platt/isotonic, C2). Validated on the n=6 golden set: ****mean F1 0.307 ? 0.423**
(+0.116)******, testlarge

79 0.000 ? 0.300, and the learned variant now ****beats CONTROL (0.388)****. Threshold-in-LOVO
is the big lever;

80 calibration is available too.

81

82 ****Next:**** a real fusion-path run to confirm Gate-A served-side; per-video output layout
83 (PER_VIDEO_OUTPUT_LAYOUT.md) so each video's artifacts are
self-contained.

84

85 **###** First end-to-end A/B on the golden corpus (n=6) ? ****ARCHIVED**** *(2026-06-14)*

86 First real use of the harness + C1?C4 helpers, 100% offline (re-scored persisted WASB
trajectories, no GPU).

87 ****Gate-A (net-crossings ? 2) is a measured win**** ? ?F1 +0.074, 6/6 matches, sign-test
****p = 0.031****, CI

88 [+0.042, +0.104], over-segmentation (segment-count ratio) ****1.68 ? 1.01****. The window-
level 5-feature logistic

89 ****did not clear the bar**** (?0.081, 2W/4L; a threshold/calibration artifact ? ***not*** the
stronger per-frame

90 prototype). The framework held up: ****C1 promoted one variant and refused another; C4**
surfaced over-segmentation

91 tIoU hid.**** Full record + tables ?**

92 [archives/quality-iterations/2026-06-first-ab-experiment/](archives/quality-
iterations/2026-06-first-ab-experiment/README.md).

93 Next: Gate-A is a candidate for the no-regression promotion gate; the logistic failure
is the C2 (calibration)

```

94     + threshold-in-loop task list.
95
96     ### Eval-honesty helpers (course corrections C1?C4) ? additive, default-OFF
97     *(2026-06-14)*
98     **Why.** A multi-agent literature review (folded into
99     [ANALYZERS_AND_RECONCILERS.md](ANALYZERS_AND_RECONCILERS.md) §13) confirmed the
100    framework's *skeleton* is
101    standard prior art (Temporal Action Localization/Segmentation + late/score-level fusion
102    ? cite, don't claim),
103    and flagged that our **measurement** is walking into known small-N traps that would make
104    "promote only on
105    measured lift" promote noise or hide a real win. Four corrections, shipped as **pure,
106    additive helpers** (no
107    existing `compare`/LOVO/`metrics.score` behaviour changed ? callers opt in):
108    - **C1 ? never a single mean.** `backend/eval/eval_stats.py`:
109    `mean_with_ci`/`bootstrap_ci` (deterministic,
110    seeded), `paired_sign_test` (the exact distribution-free "won on enough *videos*?"
111    test the corpus already
112    needs), `paired_delta_ci` (B?A with a CI that must clear 0). At n?6 a lone LOVO mean
113    is not trustworthy.
114    - **C2 ? calibrate cue confidence.** `backend/eval/score_calibration.py`: Platt +
115    isotonic (PAVA) calibrators
116    + `brier_score`/`expected_calibration_error`, so a CNN softmax, an audio onset, and an
117    integer net-crossing
118    count are comparable before the arbiter and the over-confident cue can't swamp the
119    rest.
120    - **C3 ? honest stacking.** `eval_stats.out_of_fold_predictions`/`grouped_folds` (leave-
121    one-group-out by
122    match) so a *learned* producer's outputs used as arbiter features are out-of-fold (no
123    leakage).
124    - **C4 ? over-segmentation-visible metrics.** `backend/eval/segmentation_metrics.py`:
125    `f1_at_overlaps`
126    (F1@{10,25,50}), `average_precision`/`mean_average_precision` (mAP@tIoU),
127    `segment_count_ratio`. tIoU alone
128    is blind to a rally split in 3 or two rallies merged ? exactly what the reconciler
129    exists to fix.
130
131    **Status.** Helpers + 24 unit tests landed (pure stdlib, run in the numpy/GPU-free
132    lanes); ruff + mypy clean.
133    **Not yet wired into the default harness flow** (wiring `compare`/`run_regression.py` to
134    report CIs + F1@k is
135    the follow-on, owner-gated). The discipline they encode is the methodology for every
136    future cue measurement ?
137    see §13/C1?C4 for the citations and the remaining (Tier-2/3) corrections.
138
139    ### Multi-signal fusion P2: the FusionSegmenter *(in progress, 2026-06-14)*
140    **Hypothesis.** Adjudicating shuttle-seeded windows with the net-crossing **Gate A**
141    (`min_crossings?2`)
142    and a person-occupancy **Gate B** raises **precision** on the held-shuttle / warm-up /
143    one-sided-feed /
144    spectator false positives ? *without* hurting recall, because the shuttle stays the
145    primary,
146    camera-invariant seed.
147
148    **Scope ? and an honesty boundary from the archive (read this).** The
149    [`2026-06-multivideo-lovo`](archives/quality-iterations/2026-06-multivideo-
150    lovo/README.md) iteration
151    already smoke-tested **court-region detection as a fix for the *multi-court contrast*
152    confound** and
153    found it **low-headroom**: only 12?14% of dead-time shuttle detections fall *outside*
154    the foreground
155    court, so a perfect court gate lifts contrast to only ~1.6?2.2× (below the ?3.6× "works"
156    threshold) ? the
157    multi-court problem is **temporal**, and the **learned model**, not a court mask, is its
158    answer. So Gate B

```

```

132     here is **NOT** sold as the multi-court fix. It targets a **different, previously-
133     untested** failure mode
134     (held-shuttle/warm-up/one-sided/spectator FPs). Two distinct levers; don't conflate
135     them.
136     **Data support for Gate A.** The same archive shows real rallies on the multi-court
137     matches contain
138     **?2 net-crossings 97% of the time (median 5)** while a single service feed crosses
139     ~once ? `min_crossings?2`
140     is a principled, data-backed discriminator (it's the held-shuttle owner-bug killer).
141     **Shipping shape.** A NEW `fusion` segmenter, **default OFF** (`default_segmenter`
142     unchanged); Gate A
143     wired into the live trajectory path behind `indexing.min_crossings` (default 0 = OFF ?
144     wasb/tracknet
145     byte-identical today); the court mask is an **optional** config polygon (omit ? Gate B
146     is
147     player-count-only; supply ? masked). Person features (`players_in_court,
148     two_sided_motion,
149     mean_player_motion, in_court_ratio, shuttle_player_colocation`) persisted into candidate
150     `stats` for P3's
151     learned fusion. Reuses `PersonDetector` (P1) + `rally_gate` (already existed) ? no
152     reinvention.
153     **Status (2026-06-14): SHIPPED, default-OFF.** `FusionSegmenter`
154     (`backend/pipeline/segmenters/fusion_hybrid.py`,
155     registry key `fusion_hybrid`) + the pure feature math
156     (`backend/pipeline/detectors/fusion_features.py`) + two
157     identity hooks on the trajectory engine (`_enrich_candidate_stats`,
158     `_select_for_handoff`) + `PersonDetector`.
159     detections_per_frame` + the `FusionConfig` block. Built design-first (a
160     `fusion-p2-design` fan-out workflow), then
161     **adversarially reviewed by a 31-agent workflow (25 findings ? 19 confirmed)**: the
162     headline confirmation is
163     **NO-REGRESSION verified** ? wasb/tracknet twins persist byte-identical stats and hand
164     off all windows (pinned by
165     the extended equivalence test); default `FusionSegmenter` == its wasb substrate. The
166     confirmed low/medium findings
167     were fixed (frame-dim guards, `round()` frame indices, the per-window axis-estimate
168     redundancy ? a reusable
169     `rally_gate` estimate, honest torch-import docstrings, and 6 new tests closing the
170     person-detector / Gate-B /
171     substrate / colocation-halo gaps). Suite 441?467 green**; ruff + mypy clean.
172     **Two findings deferred with intent** (reviewer-agreed): person features are computed
173     eagerly even for
174     Gate-A-pruned windows (a P3 redesign, would change the persisted-superset semantics);
175     and `fusion.velocity_threshold`
176     must be set equal to the substrate's to A/B against a tuned wasb run (documented in
177     `FusionConfig`).
178     **Lift still to be measured** offline on the 6 golden videos via
179     `experiment.compare(CONTROL, fusion)`; the
180     held-shuttle-drop + person-feature values are GPU/owner-side. *(Measured numbers
181     appended here when run ? no claims before.)*
182     ### Person cue extracted to a reusable producer (fusion P1) ?
183     [#115](https://github.com/avidullu/badminton-highlight-indexer/pull/115), 2026-06-13
184     Lifted the torchvision detector + ByteTrack + velocity windowing out of `yolo_hybrid`
185     into
186     `backend/pipeline/detectors/person_detector.py::PersonDetector`. **No behavior change**
187     (proven by the
188     unchanged `process_video` output tests); `yolo_hybrid` now orchestrates only. **Why it's
189     a quality move:**
190     it makes "add the person cue to fusion" an *add-a-producer*, not a segmenter rewrite.
191     Suite 440?441 green.
192     167

```

```

168     ### Experiment harness: A/B without rollback risk ?
[114](https://github.com/avidullu/badminton-highlight-indexer/pull/114), 2026-06-13
169     Shipped `backend/eval/experiment.py` ? the thin variant/experiment layer (the audit's
"-80% already
170     exists" held; pure glue over
`metrics`/`calibration`/`classifier`/`training`/`regression.gt_hash`). Gives
171     `Variant` (+ pinned `CONTROL`), `compare()` (labeled A/B, LOVO-honest),
**`compare_unlabeled()`** (the SxS
172     read on NEW unlabeled footage ? agreed / A-only / B-only), and a JSON experiment
leaderboard. **This is
173     the instrument every iteration is measured on.** 19 tests incl. the no-regression
invariant (all cues OFF
174     ? byte-identical to control). Suite 421?440 green.
175
176     ---
177
178     ## Archived predecessors (the historical trail)
179     Completed iterations live under [`archives/quality-iterations/`](archives/quality-
iterations/README.md):
180     - **`2026-06-initial/`** ? seed of the quality phase.
181     - **`2026-06-human-gt-tuning/`** ? first eval vs **human** GT on a partial-label video;
partial-label
182     harness + labeled-window methodology; baseline 0.650 ? tuned 0.742 (*fit-on-self*);
found+fixed a
183     negative-start ingestion bug (collector PR #13).
184     - **`2026-06-multivideo-lovo/`** ? the **n=6 corpus + honest LOVO** ; established the
heuristic is
185     **footage-fragile** (multi-court Kushagra 0.16 / gBaaddy 0.28), the **"contrast"
metric** predicts
186     heuristic F1 monotonically, and a **learned per-frame prototype overtook the heuristic
at n=3 and holds
187     ~+0.10** (n=6: learned 0.583 vs heuristic 0.480), with **AUC 0.83?0.92 decoupled from
contrast**. This
188     is the data-backed green light for the owned model.
189
190     ---
191
192     ## Standing lessons (carry-forward)
193     - **Persist detection once, re-score variants forever.** Expensive/risky = per-frame
detection (GPU/WSL);
194     cheap/swappable = the decision. Most experiments never enter the served pipeline.
195     - **LOVO or it didn't happen.** Score on a held-out *video*, never a held-out window
from the same video.
196     Group by **match**, not file. Sign test needs n ? ~n=10/9-wins for p<0.05.
197     - **The multi-court confound is TEMPORAL, not spatial/loudness** ? three heuristics were
tested and failed
198     for the same reason (foreground spatial gate **?0.04** ; audio energy **AUC ~0.6** ;
court-region
199     separability **12?14% out-of-region**): the dead-time confusion is rally-vs-non-rally
activity in the
200     *same place and frequency band*. Only the learned model's temporal/structural features
crack it (AUC
201     0.83 on Kushagra/gBaaddy). **Don't re-derive these dead ends.**
202     - **Court-masking ? multi-court fix** (see above) ? it's a **precision** tool for off-
court persons and
203     one-sided feeds, a *different* lever; measure its lift, don't assume it.
204     - **Net-crossings are a strong rally signal** ? 97% of real rallies cross ?2x (median
5); single feeds ~1x.
205     Gate A leans on this.
206     - **Small-N feature discipline.** Few, scale/translation-invariant features
(counts/ratios, not raw
207     coords) + a logistic regression, not a net ? raw coords memorize this court/camera.
208     - **Calibration, not representation, is the small-N wall** ? learned probabilities
separate (high AUC) but
209     the operating point doesn't transfer with few training videos. More distinct matches

```



```
is the highest-ROI
210     unblock.
211     - **Gate A already existed, just wasn't wired to production** ? the cheapest precision
win is plumbing, not
212     new capability. Always check what the eval drivers already do that the live path
doesn't.
213
```

13. assistant (2026-06-14T09:50:03.640Z)

Now let me read the ANALYZERS_AND_RECONCILERS doc for the signal-fusion framing, and check the distill_local feature names + segmentation_metrics + the leaderboard file to ground the design.

14. user (2026-06-14T09:50:05.033Z)

```
1      # A signal-fusion framework for rally detection ? analyzers, scopes & reconcilers
2
3      **Status:** **DESIGN ? owner-gated, NOT started.** A peer to
4      [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md) and
[MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md).
5      This doc specs a small **framework** for evolving rally-detection quality: producers
that emit **signals**
6      at three temporal **scopes** (frame / window / session), a **scorer** that *learns to
weight* them, and an
7      auditable **reconciliation** pass that lints the decision against the evidence. The goal
is to keep piling
8      on clever cues **without the pipeline becoming a pile of special cases.** Docs-only; no
implementation
9      code. Each phase below is a separate, owner-gated PR.
10
11     > **Companion docs:** the detection-layer producers (shuttle / person / audio / shuttle-
state) live in
12     > [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md) (`RallyCue` = swappable
producer; the
13     > `_enrich_candidate_stats` hook; scale/translation-invariant feature discipline, §3.2);
the no-regression
14     > A/B + LOVO measurement these ride is [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md)
(`Variant`, `compare`,
15     > `compare_unlabeled`); the scorer is the distilled logistic regression (**ADR-004**,
16     > `backend/eval/classifier.py`); the modular rally layer is **ADR-007**
17     > ([archives/decisions/DECISIONS.md](archives/decisions/DECISIONS.md)); the 2-layer
mental model is
18     > [HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md); the live quality log is
19     > [QUALITY_ITERATIONS.md](QUALITY_ITERATIONS.md).
20
21     > **Framing.** Rally detection is a genuinely hard temporal problem ? weak, multi-modal
cues; interval-level
22     > ground truth that is scarce and expensive; a confound (warm-up / knock-ups / multi-
court / held-shuttle)
23     > that looks identical to real play in any single frame or frequency band. This doc is
written as a
24     > **reusable framework**, not a badminton-specific patch: the contracts are sport-
agnostic and the design
25     > is meant to generalize to other weakly-supervised temporal-segmentation problems
(§10). Where that
26     > ambition touches research claims, §10 says plainly what is and isn't yet
substantiated.
27
28     ---
29
30     ## 1. The spine ? NO special-casing in the decision path
31
32     This is the one principle everything else serves:
33
34     > **Every producer (analyzer, detector, reconciler) emits a SIGNAL carrying a value + a
```

```

confidence/presence.
35     > The SCORING MODEL consumes those signals and *learns to weight them*. We do NOT add
hand-coded `if/else`
36     > branches into the decision.**
37
38     **Producers propose; the scorer arbitrates; the experiment harness + golden set decide
what gets promoted.**
39     That separation is what lets us add a net-hit service fault, a let-cord, a double-
bounce, a shuttle-out
40     cue, a *warm-up span*, a *shuttle-state* ? each as *"register a producer + a feature
column,"* never **add
41     a branch.** A signal that doesn't earn its keep stays at weight ? 0; it costs nothing
because it is an
42     unused column, not a live `if`.
43
44     The pattern already exists in miniature: `experiment.predict()`
(`backend/eval/experiment.py:203`) treats
45     `net_crossings` and `players_in_court` as **feature columns** ? `_gate_mask` is the
trivial scorer
46     (threshold a column), `LogisticRegression` (ADR-004) the richer learned scorer (weight
all columns), a
47     future scorer richer still. The decision path is *signals ? scorer*, with **zero** `if
service_fault:
48     drop`. Analyzers feed it more columns; reconcilers run *outside* it as an audited post-
pass.
49
50     ```
51     DETECTION ? frame-scope producers (expensive, GPU/WSL, run ONCE)
52     WASB shuttle (x,y,v,vis) · person boxes+tracks · audio hits · shuttle-
state(in_air/contact/inactive)*
53                                     ? per-frame-aligned signals (+ net estimate, window
bounds)
54                                     ?
55     shuttle-seeded candidate windows    (FusionSegmenter, P2)
56                                     ?
57     ANALYZERS ? decision-layer producers (pure, offline, re-scorable)           [forward
seam]
58     ?? window-scope (WindowAnalyzer): one candidate ? features + events + boundary
hints e.g. service-fault
59     ?? session-scope (SessionAnalyzer): whole timeline ? labeled spans + per-window
membership e.g. warm-up
60                                     ? every signal ? candidate_segments.stats as feature
columns
61                                     ?
62     SCORING / DECISION ? signals ? learned scorer (weights, NO branches)
63     _gate_mask (trivial) + LogisticRegression (ADR-004) + harness-pinned model
64                                     ? predicted rally segments
65                                     ?
66     RECONCILERS ? backward seam, "linter for rally decisions" (report-first, idempotent)
67     ConsistencyRule registry: decision-vs-evidence conflicts ? Issue + corrected set
68                                     ?
69     stats · events · spans · corrections record · highlights      EXPERIMENT HARNESS
gates EVERY promotion
70     (* shuttle-state = a richer frame detector; its PRODUCER lives in
MULTI_SIGNAL_FUSION_PLAN, consumed here ? §3.5)
71     ```
72
73     ---
74
75     ## 2. The framework ? signals, scopes, and who arbitrates
76
77     The architecture has exactly three kinds of moving part. Capability is added by
registering a producer;
78     the decision logic never grows a branch.
79

```

```

80     **Producers emit signals at three temporal SCOPES.** A signal at a coarser scope is
computed by aggregating
81     finer-scope signals; everything ultimately lands as a per-candidate feature column (or
an event/span the
82     scorer or a human consumes).
83
84     | Scope | Producer | Sees | Emits | Cost | Examples |
85     |---|---|---|---|---|---|
86     | **frame** | detector (`RallyCue`) | raw frames / audio | per-frame signal |
**expensive** (GPU/WSL) | WASB shuttle, person boxes, audio hits, **shuttle-state** (§3.5) |
87     | **window** | `WindowAnalyzer` (§3.1) | one candidate window's aligned frame-signals |
features (+conf), events, boundary hints | cheap, pure | service-fault, double-bounce, let-cord,
shuttle-out |
88     | **session** | `SessionAnalyzer` (§3.4) | the whole timeline of windows | labeled spans
+ per-window membership features | cheap, pure | **warm-up / practice**, game/break phases |
89
90     **The scorer arbitrates.** It is the only place a decision is made, and it makes it by
*weighting columns*,
91     never branching. Today: `_gate_mask` (threshold) + `LogisticRegression` (learned
weights, ADR-004). The
92     confidence column on every signal lets the scorer discount an uncertain producer instead
of trusting a
93     binary verdict ? the small-N-safe way to fold in a deterministic-ish cue.
94
95     **Reconcilers audit.** After the scorer decides, the backward seam (§6) lints the
decision against the
96     persisted evidence and proposes corrections ? report-first, never silent.
97
98     **Four invariants make this safe to grow** (the load-bearing design claims):
99     1. **Signal, not verdict** ? every producer's output is a weighted feature; the decision
path has no
100     producer-specific branch. ? unbounded cues without combinatorial special-casing.
101     2. **Expensive detection ? cheap decision** ? frame-scope detection runs once and is
persisted; window/
102     session signals and the scorer are pure re-scoring (EXPERIMENT_HARNESS §2). ?
experiments are offline,
103     deterministic, zero production risk.
104     3. **Report-first reconciliation** ? corrections are recorded and reviewable, never
applied silently. ?
105     human-in-the-loop, auditable.
106     4. **Promote only on measured held-out lift** ? LOVO on the golden set + the
`run_regression.py` gate;
107     default OFF; rollback = config flip. ? no-regression evolution.
108
109     ---
110
111     ## 3. The forward seam ? injectable analyzers
112
113     Pluggable producers ? registered like `segmenter_registry`
(`backend/pipeline/segmenters/base.py:35`) ?
114     that run over the **per-frame-aligned signals already produced**: the WASB shuttle
trajectory
115     (`frame.visible.x/.y` + velocity), the person boxes per frame
(`PersonDetector.detections_per_frame`,
116     `person_detector.py:256`), the net line (`rally_gate.estimate_play_axis`, camera-
agnostic), and the window
117     bounds. Analyzers are **pure** ? no I/O, no torch/cv2 ? mirroring `fusion_features.py`'s
discipline, so
118     their math is unit-testable in the GPU-free dev container.
119
120     ### 3.1 Contracts (JSON-serializable; proposed `backend/pipeline/analyzers/`)
121
122     ```python
123     @dataclass(frozen=True)
124     class Signal:
                                     # one scalar cue + how sure the producer is ?

```

```

never a verdict
125     value: float; confidence: float = 1.0    # persisted as `<producer>__<key>` (+
`_confidence`), [0,1]
126
127     @dataclass(frozen=True)
128     class AnalyzerEvent:                      # ? the `events` table on confirmation (§5)
129         type: str; timestamp: float; confidence: float = 1.0    # "net_hit"/"double_bounce"?
; src-absolute s
130         player: Optional[str] = None; details: Dict[str, Any] = field(default_factory=dict)
131
132     @dataclass(frozen=True)
133     class BoundaryHint:                        # OPTIONAL deterministic proposal ? never auto-
applied
134         kind: Literal["start", "end"]; timestamp: float; confidence: float; reason: str
135
136     @dataclass(frozen=True)
137     class AnalyzerResult:
138         signals: Dict[str, Signal] = field(default_factory=dict)
139         events: List[AnalyzerEvent] = field(default_factory=list)
140         boundary_hints: List[BoundaryHint] = field(default_factory=list)
141
142     @dataclass
143     class WindowSignals:                       # read-only pre-produced bundle (no I/O inside
the analyzer)
144         start: float; end: float; fps: float; frame_width: float; frame_height: float
145         shuttle: List["TrajectoryPoint"]      # in-window points (.frame/.visible/.x/.y)
146         persons: Dict[int, List[Tuple[int, BBox]]] # frame_idx ? [(track_id, box)]; {} when
person cue OFF
147         net: Tuple[str, float]                # (axis, position): rally_gate auto-estimate or
config
148         frame_states: Dict[int, "ShuttleState"] # frame_idx ? state; {} until the
shuttle-state detector (§3.5)
149         base_stats: Dict[str, float]          # motion stats already computed for the window
150
151     class WindowAnalyzer(ABC):
152         name: str; version: str                # registry key + column prefix; version ?
provenance on change
153         feature_keys: List[str]; event_types: List[str]    # declared columns/events (for
the harness)
154         @abstractmethod
155         def analyze(self, w: WindowSignals) -> AnalyzerResult: ...
156     ...
157
158     The three roles an `AnalyzerResult` can fill:
159
160     | Role | Sink | Consumed by |
161     |---|---|---|
162     | *(a) features* (with confidence) | `candidate_segments.stats` as `<analyzer>__<key>`
(+ `_confidence`) | the scoring model **and** the harness, as feature columns (§4) |
163     | *(b) events* | the `events` table (buffered on the candidate, flushed on
confirmation, §5) | history / highlights / audit |
164     | *(c) boundary hints* (optional) | stashed in `stats`; never auto-applied | the
reconciler (§6) / a future boundary-aware scorer |
165
166     ### 3.2 Worked example ? the SERVICE-FAULT analyzer (window-scope)
167
168     The owner's held-shuttle bug has a sibling: a **service fault** where the serve hits the
net and the
169     shuttle comes to rest in the net region. Pure trajectory, GPU-free, no person cue
needed:
170
171     - **Input:** the window's shuttle trajectory + the `rally_gate` net estimate.
172     - **Logic:** the shuttle **enters the net region** (`|coord ? net| < deadband`) **and**
its velocity
173     **decays to ? 0** there (comes to rest) ? high-probability net-hit service fault.

```

Confidence scales with

```

174     how cleanly the halt sits inside the net band.
175     - **Emits:** a `service_fault` **signal** (value = presence, confidence = halt
cleanliness) + a `net_hit`
176     **event** at the halt timestamp + a rally-**END** `BoundaryHint` at that timestamp.
177
178     Crucially: the analyzer does **not** drop the window. It *proposes* `service_fault`; the
scorer sees
179     `service_fault` + `service_fault_confidence` as two more columns and **learns**
whether/how much to
180     down-weight. The boundary hint is an audited proposal the reconciler may act on ? never
a branch.
181
182     ### 3.3 Injection point ? generalize `_enrich_candidate_stats`
183
184     The FusionSegmenter hook (`fusion_hybrid.py:70`) already turns "compute fixed features"
into per-window
185     enrichment. We generalize it to **run each enabled analyzer** ? additively, so the
existing
186     `net_crossings`/person paths are unchanged:
187
188     ```python
189     def _enrich_candidate_stats(self, cw, points, fps, frame_width, video_path, idx_cfg):
190         stats = super()._enrich_candidate_stats(...)          # base motion keys
191         stats["net_crossings"] = ...                          # Gate-A signal (kept)
192         # ? person features when the person cue is on (kept) ?
193         w = self._window_signals(cw, points, fps, frame_width, video_path, idx_cfg)
194         for analyzer in analyzer_registry.enabled(idx_cfg):   # NONE enabled ? byte-identical
to control
195             res = analyzer.analyze(w)
196             for key, sig in res.signals.items():
197                 stats[f"{analyzer.name}__{key}"] = sig.value
198                 stats[f"{analyzer.name}__{key}_confidence"] = sig.confidence
199                 stats.setdefault("_events", []).extend(e.__dict__ for e in res.events)      #
flushed in Phase 4
200                 stats.setdefault("_boundary_hints", []).extend(h.__dict__ for h in
res.boundary_hints)
201             return stats
202     ```
203
204     Flag-gated, **default OFF** via the existing `indexing.fusion.cue_flags` (a sibling
`analyzer_flags`
205     map keyed by `analyzer.name`). With every analyzer OFF, the loop is empty ? candidate
rows are
206     byte-identical to the substrate ? the no-regression invariant holds (§7).
`net_crossings` and the person
207     features are the **prototype analyzers** the seam retrofits first (proving it with zero
behavior change);
208     every new cue after that is a pure add.
209
210     ### 3.4 Session-scope analyzers ? the WARM-UP / PRACTICE example
211
212     Some structure is invisible at the window scope. **warm-up** looks exactly like a rally
in any single
213     window ? shuttle in flight, two players, net crossings, sustained motion ? which is
*why* it is a notorious
214     false-positive source. It is only distinguishable by **timeline structure**: a long
contiguous active
215     stretch *without* the rally?dead-time?rally cadence of scored play, usually (not always)
at the start.
216
217     So warm-up needs a producer that sees the **whole sequence of windows**, not one window.
That is a sibling
218     tier ? a `SessionAnalyzer` that runs once per video (a second pass, after per-window
enrichment) and emits

```

```

219     a labeled **span** plus a per-window **membership feature**:
220
221     ```python
222     @dataclass(frozen=True)
223     class Span:
224         kind: str                # "warmup" | "practice" | "break" | "game" | ?
225         start: float; end: float; confidence: float; reason: str
226
227     @dataclass(frozen=True)
228     class SessionResult:
229         spans: List[Span] = field(default_factory=list)
230         # window_index ? signals merged back into THAT window's stats (e.g. {"in_warmup":
Signal(0.8, 0.6)})
231         window_features: Dict[int, Dict[str, Signal]] = field(default_factory=dict)
232
233     class SessionAnalyzer(ABC):
234         name: str; version: str
235         span_kinds: List[str]; feature_keys: List[str]
236         @abstractmethod
237         def analyze(self, windows: List[WindowSignals], session: "SessionContext") ->
SessionResult: ...
238     ```
239
240     **`WarmupAnalyzer`** (worked):
241     - **v0 (free, no new model): ** find the **leading contiguous high-activity span** with
low inter-window
242     dead-time and no serve-reset cadence, over the persisted window timings we already
have. Emit
243     `Span(kind="warmup", ?)` + per-window `in_warmup` = overlap fraction of the window
with the span.
244     - **v1 (learned): ** fed by the **shuttle-state cadence** (§3.5) / the **audio thwack**
cadence ? warm-up =
245     continuous hits *without* the point-ending pause rhythm; generalizes to mid-session
practice / between-
246     game knock-ups (drop the position prior).
247
248     **You framed the consumption exactly right: "so the scorer may consume it."**
`in_warmup` is a **feature
249     column**; the scorer *learns* to down-weight windows inside the span. **Warm-up is the
poster child for
250     no-special-casing**: the naive version is `if t < warmup_end: drop` ? a hand-coded
branch hanging off a
251     fuzzy boundary; a 20-second error **deletes a real opening rally**. As a weighted
feature, a wrong boundary
252     just slightly mis-weights a couple of windows the other cues rescue. The span is *also*
persisted as
253     evidence a reconciler (§6) may use to down-weight a whole cluster, but the primary path
is feature ? scorer.
254
255     **It is measurable today with no new labels:** the golden labels already treat warm-up
knock-ups / pre-serve
256     as **dead time = negatives** (Roora starts a rally at serve-contact; fusion §1 names
"warm-up knock-ups" as
257     a precision target). So an `in_warmup` feature is directly A/B-able on the 6 golden
videos for **precision**
258     lift ? that is the promotion gate.
259
260     ### 3.5 Consuming a richer detector ? SHUTTLE-STATE (frame-scope producer)
261
262     A natural new **detector**: a per-frame shuttle-state classifier ? `in_air` /
`racket_contact` /
263     `inactive`. Note the scope: it **processes frames ? a new per-frame signal**, like WASB,
so it is a
264     **frame-scope producer in the detection layer** ? its spec belongs in
265     [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md) (it is a new `RallyCue`),

```

```

**not** a `WindowAnalyzer`.
266     This doc records **how the framework consumes it** once it exists:
267
268     - a **`WindowAnalyzer`** turns the persisted per-frame state into window features ?
`racket_contact_count`
269     (? shot cadence), `inactive_ratio` (? dead-time) ? that flow into the scorer as
columns;
270     - a **`SessionAnalyzer`** (warm-up, §3.4) reads its cadence;
271     - the **reconcilers** (§6) get *stronger* evidence: `trailing_dead_time` / `over_merged`
currently infer
272     inactivity from a velocity threshold; a learned `inactive` state is a more direct,
occlusion-robust
273     evidence source for "truncate / split here."
274
275     **Stage it (don't build the model first).** Most of the state is derivable for free from
signals we already
276     have ? `in_air` vs `inactive` ? WASB velocity + visibility; `racket_contact` ?
trajectory **direction
277     reversals** (fusion §4: "each reversal ? a shot") **and** the scaffolded **audio
thwack**
278     (`backend/pipeline/audio/hit_detector.py`, ADR-007's #1 cue). So **v0 = a derived
`shuttle_state` analyzer**
279     over existing signals (no new model); measure it. **The dedicated per-frame vision model
earns its keep
280     only where v0 hits a ceiling: WASB trajectory gaps** ? occlusion, motion blur, fast
smashes ? where there
281     is no `(x,y)` to threshold but an appearance model could still call `racket_contact`.
**Label blocker:** we
282     have **zero** per-frame state labels (golden labels are rally intervals); a dedicated
model needs weak/
283     distilled labels or a new (minors-gated) annotation task ? the same gate as feature-
level fusion. If you can
284     weak-label state from existing signals, prefer the derived analyzer until the gap-
filling case is proven.
285
286     ---
287
288     ## 4. How signals reach the scorer ? columns, not branches
289
290     The persisted `stats` columns are exactly what the harness already lifts.
`load_video_candidates(...,
291     feature_names=[...])` (`experiment.py:429`) reads each name out of the row's `stats`
JSON into a feature
292     column; `predict()` (`experiment.py:203`) gates/scores on those columns and merges. So a
new producer's
293     signal becomes a scorer input by **adding its column name to a `Variant`'s
`feature_names`** ? no new
294     scoring code, no branch:
295
296     ```python
297     # A learned Variant that lets the scorer weight the new signals (service-fault + warm-up
membership):
298     fusion_plus = Variant(name="fusion+analyzers",
299                           feature_names=[*SHUTTLE_FEATURES, "person__players_in_court",
300                                           "service_fault__service_fault",
301                                           "service_fault__service_fault_confidence",
302                                           "warmup__in_warmup",
303                                           "warmup__in_warmup_confidence"])
304     compare(golden_videos, CONTROL, fusion_plus)          # LOVO-honest B ? A on the 6 golden
videos
305     ...
306
307     The model **learns the weights** ? down-weight a window with a high-confidence
`service_fault` or high
308     `in_warmup`; up-weight ?2 in-court players + multiple net-crossings. **What the scorer

```

```

cannot see, it
307     cannot special-case; what it can see, it weights from data.** Same logistic regression
as ADR-004; we add
308     columns, never a net.
309
310     ---
311
312     ## 5. Persistence
313
314     - **Features ? `candidate_segments.stats`** (`database.py:240`). Namespaced
`<producer>__<key>`
315     (+ `_confidence`). Persisted **as a shadow** even when the served decision ignores
them, so any future
316     `Variant` re-scores offline (EXPERIMENT_HARNESS §5.2). `version`/`name` recorded for
provenance.
317     - **Spans ? window membership + a session record.** A `SessionAnalyzer`'s per-window
`in_<span>` features
318     ride in each window's `stats` (so they re-score like any feature); the span list
itself (start/end/kind/
319     confidence) is a small per-video record (a JSON sidecar first; a table only if it
grows).
320     - **Events ? the `events` table** (`database.py:227`, `add_event(segment_id, ts, type,
player, details)`).
321     Its FK is to `play_segments`, which exist only **after** the AI handoff confirms a
candidate. So
322     analyzer events are **buffered on the candidate** (`stats["_events"]`) and **flushed**
to `events` in
323     Phase 4, when `_candidate_id` ? `segment_id` is linked (`trajectory_hybrid.py:283`).
Candidate-level
324     discoveries survive for offline re-scoring; the `events` table stays "events on
confirmed segments."
325     - **Corrections record** (§6) ? report-first means corrections are **recorded, never
silent.** Phase one:
326     a JSON sidecar shaped like `experiment_leaderboard.json` (one row per issue: segment,
kind, evidence,
327     before?after, reason, timestamp). When auto-apply is promoted, add a `corrections`
table via the
328     `PRAGMA user_version` migration ladder (`database.py:88`, `if version < 3:`) so
applied corrections are
329     queryable and reversible.
330
331     ---
332
333     ## 6. The backward seam ? the reconciliation pass (a linter for rally decisions)
334
335     Given **predicted segments + the persisted per-frame/per-window/per-session signals**,
detect where the
336     **DECISION contradicts the EVIDENCE** and propose corrections. This is the backward dual
of the analyzers:
337     analyzers add evidence *before* the decision; reconcilers audit the decision *against*
evidence *after* it.
338
339     ### 6.1 Contracts (proposed `backend/pipeline/reconcilers/`)
340
341     ```python
342     @dataclass(frozen=True)
343     class Issue:
344         segment_index: int; kind: str          # kind = rule id, e.g. "trailing_dead_time"
345         evidence: Dict[str, Any]              # the signal values that triggered it
346         before: Interval; after: Optional[List[Interval]] # None=drop · [iv]=corrected ·
[iv1,iv2]=split
347         reason: str; confidence: float
348
349     @dataclass(frozen=True)

```



```

350     class Correction:
351         issues: List[Issue]; corrected: List[Interval]      # the FULL corrected set
(idempotent)
352
353     class ConsistencyRule(ABC):
354         name: str; precedence: int          # FIXED ordering; lower runs first
355         @abstractmethod
356         def check(self, seg: Interval, signals: "SegmentSignals") -> Optional[Issue]: ...
357
358     def reconcile(segments, signals, rules) -> Correction: ... # fixed precedence;
idempotent
359     ```
360
361     ### 6.2 Worked rules (fixed precedence)
362
363     | # | Rule | Evidence contradiction | Correction |
364     |---|---|---|---|
365     | 1 | `trailing_dead_time` | segment still "in rally" at its end, but the shuttle went
inactive earlier | truncate to the last-active frame *(the owner's misfire fix)* |
366     | 2 | `no_net_crossing` | `net_crossings < 1` ? nothing ever crossed the net | drop
(or flag) |
367     | 3 | `over_merged` | a dead-time gap > `merge_gap` of shuttle-inactivity sits
inside one segment | split at the gap |
368     | 4 | `boundary_slack` | first sustained shuttle activity / first net-crossing is well
inside the start | tighten start to serve-contact |
369     | 5 | `inside_warmup` | the segment lies inside a high-confidence `warmup`/`practice`
span (§3.4) | flag (drop only under apply-flag) |
370
371     Each rule reads only persisted evidence (trajectory activity per frame,
`net_crossings`, person
372     occupancy, analyzer signals/events, session spans, shuttle-state) ? it is a
deterministic linter, not a
373     re-detector. As richer producers land (shuttle-state, warm-up), rules get stronger
evidence, not new
374     branches in the decision path.
375
376     ### 6.3 Report-first, apply-optional (non-negotiable)
377
378     `reconcile()` returns both an issues report and a corrected set. It NEVER
silently rewrites
379     (auditability + the repo's honesty/attribution working agreement). The pipeline either:
380
381     - (default) persists the report and surfaces it for human review ? production
output unchanged; or
382     - (flagged, `reconcile.apply`) applies the corrected set, recording every change
in the
383     corrections record (before?after, rule, evidence). Rollback = flip the flag.
384
385     It is a `Variant` transformation, so its lift is A/B-measurable BEFORE auto-apply. A
reconciler is a
386     pure post-`predict` stage: `reconcile(predict(...), signals, rules).corrected`. Wrapping
that as a
387     RECONCILED variant lets `compare(CONTROL, RECONCILED)` report the LOVO lift on the 6
golden videos
388     ([QUALITY_ITERATIONS.md](QUALITY_ITERATIONS.md) corpus) ? promote to auto-apply only
on measured lift.
389
390     Idempotent; fixed rule precedence. `reconcile(reconcile(x)) == reconcile(x)`
(truncating an
391     already-truncated segment is a no-op); precedence is a fixed table so the result is
deterministic and
392     order-independent of input segment order.
393
394     ---
395

```

```

396     ## 7. No-regression discipline (identical to the harness contract)
397
398     The same four guarantees EXPERIMENT_HARNESS.md §6 makes ? applied to every producer and
the reconciler:
399
400     - **Default OFF.** Analyzers gate on `analyzer_flags` (default empty); the reconciler is
report-first
401     (apply gate default off). Merged code **cannot** change served behavior until
explicitly enabled.
402     - **Shadow-persist, then promote.** Signals/spans/issues are persisted even while
ignored, so a `Variant`
403     re-scores them offline; a signal becomes a *decision input* only when a Variant that
references it wins
404     LOVO. (EXPERIMENT_HARNESS §5.2's shadow mode.)
405     - **Promotion only via the eval gate.** `run_regression.py` against the golden
`baseline.json` (exit 1
406     blocks); the experiment leaderboard records the spec hash + per-video + LOVO deltas +
label-set hash.
407     - **Rollback = flip a config flag, never `git revert`.** The pinned `CONTROL` variant is
always present.
408
409     The keystone test (extends the harness's existing invariant): **all analyzer flags OFF +
reconciler
410     report-only ? candidate rows AND predictions byte-identical to control.**
411
412     ---
413
414     ## 8. How it composes with fusion P2
415
416     - **Window-analyzers plug into fusion P2's enrichment hook.**
`FusionSegmenter._enrich_candidate_stats`
417     (`fusion_hybrid.py:70`) is the exact injection point §3.3 generalizes; `net_crossings`
and the person
418     features become the first registered analyzers (no behavior change). New cues are add-
a-producer, per
419     ADR-007 / MULTI_SIGNAL_FUSION_PLAN §2.
420     - **Session-analyzers run as a second pass** over the per-window stats the same hook
persists.
421     - **The shuttle-state detector is a new fusion `RallyCue`** (frame-scope producer) ? it
belongs in the
422     fusion plan; this framework only *consumes* it (§3.5).
423     - **Reconcilers consume fusion P2's persisted signals** ? the same
`candidate_segments.stats` plus the
424     persisted trajectory; no new detection.
425     - Everything inherits fusion P2's control-reproducing defaults, so the fusion segmenter
with all flags off
426     still seeds and hands off identically to `wasb_hybrid`.
427
428     ---
429
430     ## 9. Trade-offs & risks
431
432     - **Deterministic vs learned signals.** A producer may emit a verdict-shaped signal
(`service_fault=1.0`),
433     but we persist it as a **feature**, so the scorer still arbitrates. Risk: a
confidently-wrong cue.
434     Mitigation: the **confidence column** + the **LOVO gate** ? a bad signal earns ? 0
weight, never promotes.
435     - **The temporal confound is the hard core.** Warm-up-vs-play, multi-court, and held-
shuttle all look
436     identical in any single frame / frequency band ? QUALITY_ITERATIONS' standing lesson:
"the dead-time
437     confusion is rally-vs-non-rally activity in the same place and frequency band... only
the learned model's
438     temporal features crack it." This is *why* warm-up is a **feature the model weights**,

```

```

not a heuristic
439     span-cutter, and why session-scope structure earns its own tier.
440     - **Scope discipline.** Put a producer at its *natural* scope: shuttle-state is frame-
scope (a detector),
441     not a window analyzer; warm-up is session-scope, not per-window. Mis-scoping forces a
producer to either
442     recompute detection or miss the structure it needs.
443     - **Ordering / idempotence (reconcilers).** Fixed precedence table + an idempotence test
444     (`reconcile?reconcile == reconcile`).
445     - **Signal, not verdict.** A producer that *decides* (returns keep/drop) re-introduces
special-casing;
446     reviews must reject "analyzer returns a boolean keep/drop."
447     - **Report-first auditability.** Reconcilers never silently mutate output; every applied
change is recorded.
448     - **Small-N feature discipline.** Few, scale/translation-invariant columns
(counts/ratios/presence, never
449     raw coords ? MULTI_SIGNAL_FUSION_PLAN §3.2); the model stays a logistic regression,
not a net.
450     - **Labels gate the learned producers.** Shuttle-state and feature-level fusion need
labels we don't yet
451     have; warm-up is measurable *now* because golden treats it as negatives. Know which
producers are gated.
452     - **Cost & licensing.** Pure analyzers are GPU-free; person- and shuttle-state-dependent
producers pull
453     torch / a second WSL pass ? gate behind their cue flags; all deps stay MIT/Apache/BSD
(ADR-002/005).
454     - **Event-table timing.** The buffer-then-flush wiring (§5) is the one non-obvious
integration detail.
455
456     ---
457
458     ## 10. Generality & research positioning
459
460     This is written as a **framework**, not a badminton patch. Read sport-agnostically it
is: *a registry-based,
461     multi-scope **signal-fusion** architecture for **weakly-supervised temporal
event/segment detection**, with
462     a learned arbiter and an **auditable post-hoc correction** layer.* The pieces that
generalize:
463
464     - **Multi-scope producers ? one feature table ? a learned arbiter.** The
frame/window/session taxonomy and
465     the "signal-not-verdict" rule are domain-independent. Any temporal problem with
**weak, multi-modal cues
466     at different time scales** (broadcast sports event spotting, surgical-phase
recognition, call-center /
467     meeting segmentation, activity logs) fits the same shape: add a producer, add a
column, let the arbiter
468     weight it, gate promotion on held-out lift.
469     - **Detection ? decision (persist once, re-score forever).** Decouples the expensive,
hard-to-reproduce
470     perception from the cheap, deterministic decision ? the property that makes the whole
thing
471     experimentally honest and cheap to iterate.
472     - **Auditable reconciliation as a first-class layer.** Most pipelines bury post-hoc
fixes as silent
473     if-branches; treating consistency-vs-evidence as a *report-first, A/B-measured* pass
is the transferable
474     methodological idea (and the human-in-the-loop / safety story).
475     - **The honest measuring stick.** Interval-level, scarce ground truth + **leave-one-
VIDEO-out** (group by
476     match, never a held-out window from the same video) is the discipline any small-corpus
temporal study
477     needs to avoid flattering itself.
478

```

479 **What is defensibly contributed vs. what is standard.** Multi-modal **late fusion**,
 weak supervision, and
 480 learning a linear/logistic arbiter over hand-designed features are all standard. The
 combination offered
 481 here ? (a) a uniform producer-registry across **three temporal scopes**, (b) a hard
 no-branch decision
 482 invariant that makes extension monotone, (c) a **report-first, harness-measured
 reconciliation** layer, and
 483 (d) the **detection?decision persistence** substrate that makes every experiment a zero-
 risk offline
 484 re-score ? is what would be written up, with rally detection as the motivating hard
 problem.
 485
 486 **Honesty caveat (attribution working agreement).** No external results or citations are
 asserted here. A
 487 real write-up still owes: a proper **related-work survey** (temporal action
 segmentation/detection, sports
 488 event spotting, multimodal fusion, weak supervision ? to confirm which claims are
 actually novel), **strong
 489 baselines**, **per-producer ablations**, and reporting on **>1 sport / camera** to
 substantiate the
 490 generality claim. Today the corpus is **6 matches** (QUALITY_ITERATIONS) ? enough for
 direction-checks and
 491 LOVO sign-tests, **not** enough to claim a general model; corpus growth is the highest-
 ROI unblock. Treat
 492 §10 as a *positioning sketch*, not a results section.
 493
 494 > **Update (2026-06-14): that related-work survey is now done ? see [§13](#13-prior-art-
 terminology--course-corrections-literature-review-2026-06-14).**
 495 > The honest finding: the *skeleton* (TAL/TAS + late fusion) is standard and must be
 cited, not claimed;
 496 > the only `novel-combination` is the report-first reconciler-as-linter bundle; warm-up-
 as-a-soft-feature is
 497 > the thinnest genuine gap. §13 also lists the verified citations and the prioritized
 course corrections.
 498
 499 ---
 500
 501 ## 11. Phased execution (owner-gated; ONE PR per slice, off `master`, default OFF)
 502
 503 - **A0 ? this doc**, indexed in `docs/README.md`, cross-linked from the peers + ADR-007.
 *(this
 504 deliverable; docs-only.)*
 505 - **A1 ? analyzer contracts + registry**
 (`WindowAnalyzer`/`AnalyzerResult`/`Signal`/`AnalyzerEvent`/
 506 `BoundaryHint` + `analyzer\_registry`); generalize `\_enrich\_candidate\_stats` to iterate
 enabled analyzers
 507 (none enabled ? byte-identical). Retrofit `net\_crossings` + person features as the
 first two analyzers.
 508 - **A2 ? service-fault analyzer** behind `analyzer\_flags.service\_fault` (default OFF);
 signal + `net\_hit`
 509 event + end boundary hint; shadow-persisted; offline `compare(CONTROL,
 +service_fault)` LOVO.
 510 - **A3 ? event flush wiring** (buffer analyzer events on candidates ? `add\_event` on
 confirmation).
 511 - **A4 ? `SessionAnalyzer` tier + `WarmupAnalyzer` v0** (leading-span heuristic over
 window timings),
 512 default OFF; persists `warmup\_\_in\_warmup` + the span record; measured for
 precision lift on golden.
 513 - **A5 ? derived `shuttle\_state` analyzer v0** (velocity + visibility + direction-
 reversals (+ audio);
 514 **no new model**); emits `racket\_contact\_count` / `inactive\_ratio`; measured LOVO.
 *(The dedicated
 515 per-frame shuttle-state DETECTOR is a separate MULTI_SIGNAL_FUSION_PLAN `RallyCue`
 item ? gated on

```

516     per-frame labels; build only if v0 is blocked by trajectory gaps.)*
517 - **R1 ? reconciler contracts + registry + `reconcile()`**
518   (`ConsistencyRule`/`Issue`/`Correction`) with
519   the worked rules; pure, idempotent, unit-tested; **not** wired to apply (report-only).
520 - **R2 ? reconciler as a `Variant` transform**: optional post-`predict` stage;
521   `compare(CONTROL,
522     RECONCILED)` LOVO; report-only in production.
523 - **R3 ? corrections persistence + opt-in apply**: JSON sidecar first, then the
524   `corrections` table
525   (`user_version < 3`); apply behind `reconcile.apply`; **rollback = flip the flag.**
526
527   Each PR is gated by the no-regression invariant ($7) + `run_regression.py`. Order is a
528   suggestion; A4/A5/R1
529   are independent and can be reprioritized on owner steer.
530
531   ---
532   ## 12. Verification
533
534   - **Offline, runnable in this dev container (no GPU/torch/cv2):** pure analyzer math on
535     synthetic
536     trajectories + person boxes (service-fault fires on a synthetic net-halt, silent on a
537     clean rally;
538     `WarmupAnalyzer` v0 finds a synthetic leading knock-up span; derived `shuttle_state`
539     matches a scripted
540     velocity/visibility sequence); the **no-regression invariant** (all flags OFF +
541     reconciler report-only ?
542     candidate rows + predictions byte-identical to control); the **reconciler
543     idempotence** test
544     (`reconcile?reconcile == reconcile`) and fixed-precedence determinism;
545     `compare(CONTROL, variant)`
546     deterministic on synthetic candidate rows.
547   - **Metrics ? adopt the field's (see [$13](#13-prior-art-terminology--course-
548     corrections-literature-review-2026-06-14)/C4):**
549     report **segmental F1@{10,25,50} + edit score** (TAS) and **tight/loose mAP@?**
550     (action spotting) alongside
551     temporal-IoU. tIoU alone is nearly blind to the over-segmentation the reconciler
552     exists to fix, so a
553     reconciler win ? or an IoU-chasing rule that shreds rally structure ? is invisible
554     under tIoU.
555   - **Machine-side (owner, GPU/WSL ? full pipeline can't run here):** persist analyzer
556     signals on a real
557     clip; `compare(CONTROL, +service_fault)`, `compare(CONTROL, +warmup)`,
558     `compare(CONTROL, RECONCILED)`
559     LOVO on the 6 golden videos; confirm the owner's **trailing-dead-time misfire**
560     truncates, the
561     **held-shuttle / net-hit service feed** is down-weighted, and **warm-up windows** lose
562     precision-mass;
563     spot-check `net_hit` events on confirmed segments. **Grade with $13/C4 metrics + per-
564     fold CIs (C1).**
565   - **Docs (A0):** links resolve; indexed in `docs/README.md`; cross-linked from
566     [MULTI_SIGNAL_FUSION_PLAN.md](MULTI_SIGNAL_FUSION_PLAN.md),
567     [EXPERIMENT_HARNESS.md](EXPERIMENT_HARNESS.md),
568     [HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md), and ADR-007.
569
570   ---
571
572   ## 13. Prior art, terminology & course corrections (literature review, 2026-06-14)
573
574   A multi-agent web-research pass surveyed the eight design patterns of this framework
575   against the
576   literature and **adversarially verified every citation** (real-paper check + closer-
577   prior-art hunt).
578   **Result: zero fabricated papers**; a few wrong author-lines / metadata slips were
579   caught and are

```

558 corrected below. This section is the honest "are we reinventing the wheel?" answer and the actionable

559 course corrections ? all ****owner-gated, NOT yet implemented**** (they fold into the A*/R* phases of §11).

560

561 **### 13.1 Verdict ? what is standard vs. what is ours**

562

563 > ****TL;DR.**** The architecture's **skeleton** is standard and must be ****cited, not claimed****: candidate-windows

564 > + learned scorer = ****Temporal Action Localization**** (proposal-then-score); the reconciler's

565 > over-segmentation fixes = ****Temporal Action Segmentation**** refinement/post-processing; "signal not verdict /

566 > learned arbiter over cue columns" = ****late / score-level fusion**** realized as ****stacked generalization**** ?

567 > ~20 years old and **already present in racket sports**. The ****defensible delta** is the engineering/governance

568 > envelope²: detection?decision persisted as a re-score substrate + a ****report-first, A/B-lift-gated,**

569 > idempotent, human-in-the-loop reconciler-as-linter² keyed on decision-vs-evidence **contradiction** + the

570 > flag-gated no-regression promotion harness, on a deliberately tiny LOVO-by-match golden set with an

571 > interpretable scorer. Of the patterns, only the ****reconciler-as-linter**** rated *`novel-combination`*; the

572 > rest are *`partial-overlap`*. Park any novelty claim narrowly on (a) that reconciler bundle and (b)

573 > ****warm-up-as-a-soft-session-feature**** (the one slice with genuinely thin prior art).

574

575 | Pattern (our framing) | Field's name | Closest verified prior art | Verdict |

576 |---|---|---|---|

577 | Candidate-windows + scorer + reconciler-as-refinement | Temporal Action Localization / Segmentation + boundary refinement | ASRF (arXiv:2007.06866); Activity Grammars (arXiv:2312.04266); BMN (arXiv:1907.09702) | partial-overlap |

578 | "Signal not verdict" ? learned arbiter over cue columns | Late / score-level fusion + stacked generalization | Snoek 2005 (10.1145/1101149.1101236); Wolpert 1992; racket-sports A/V rally-break fusion (CVIU 2009) | partial-overlap |

579 | Detection?decision, "persist once, re-score forever" | Frozen-backbone cached features + decoupled proposal/classification | S-CNN (arXiv:1601.02129); CBR (arXiv:1705.01180); Decouple-SSAD (arXiv:1904.07442) | partial-overlap (? field default) |

580 | Report-first reconciler (contradiction ? audited correction, measured before apply) | Model assertions + constraint-based repair ("ML linter") | Model Assertions (arXiv:2003.01668); HoloClean (arXiv:1702.00820); TFDV (MLSys 2019); EventAnchor (arXiv:2101.04954) | ****novel-combination**** |

581 | Scarce interval labels + tiny LR + LOVO-by-match | Point/timestamp-supervised TAL + grouped (LOGO/LOSO) CV | SF-Net (arXiv:2003.06845); L00CV-bias (arXiv:2406.01652); Snorkel (arXiv:1711.10160) | partial-overlap |

582 | Multi-scope (frame/window/session); warm-up conditions per-window | Multi-scale/hierarchical modeling + play-break + phase recognition | Tjondronegoro ICME 2003; GL-MSTCN (10.1186/s12938-022-01048-w); MS-TCN (arXiv:1903.01945) | partial-overlap |

583

584 ****In-domain prior art that already hard-codes our cues**** (so our delta is the **no-branch discipline**, not the

585 cues): non-broadcast badminton rally start/end (See et al. 2024/2025), shuttle-state/hit detection (MonoTrack

586 arXiv:2204.01899; Hsu/Yu/Cheng, Sensors 2024 24(13):4372), service-fault (Goh et al., Sensors 2023 23(24):9759),

587 tennis in-play state machines (10.1080/19346182.2013.819007), badminton point-segmentation (Ghosh et al. 2018,

588 arXiv:1712.08714), ShuttleSet (arXiv:2306.04948).

589

590 **### 13.2 Terminology to adopt** (gloss our nouns with the field's on first use)

591

592 | Our term | Field's established term | Anchor |

593 |---|---|---|

```

594 | `RallyCue` / "producer" | **labeling function** / weak-supervision source; **base
model** in a stack | Snorkel; Wolpert 1992 |
595 | "Signal not verdict / no special-casing" | **late / score-level fusion**; arbiter =
**level-1 combiner / meta-learner** | Snoek 2005; Wolpert 1992 |
596 | ADR-004 LR scorer (`backend/eval/classifier.py`) | **stacked meta-learner / level-1
combiner** | Wolpert 1992 |
597 | "Detection?decision / persist once, re-score" | **frozen-backbone cached features**;
**decoupled proposal/classification**; offline re-scoring | S-CNN; Decouple-SSAD |
598 | `WindowAnalyzer` (window-scope) | **temporal action proposal** features /
**actionness** | S-CNN; BMN |
599 | Session-scope / warm-up | **video-level temporal context / phase segmentation**;
**play-break** outer layer; **global-local** | Tjondronegoro 2003; GL-MSTCN |
600 | `ConsistencyRule` reconciler | **model/consistency assertions**; **constraint-based
repair**; **boundary refinement**; **activity grammar / constrained decode** | Kang 2020;
HoloClean; ASRF; Activity Grammars |
601 | "Report-first linter" | **data validation / schema anomalies** ("unit tests for data")
| TFDV; Deequ |
602 | "Over-merged / split" . "tighten boundary slack" | **over-segmentation error** .
**boundary regression/refinement** | MS-TCN; ASRF; CBR |
603 | "LOVO, group by match" | **leave-one-group-out / -subject-out (LOGO/LOSO)** grouped CV
| sklearn `GroupKFold`; Austin 2025 |
604 | Temporal-IoU metric | **mAP@tIoU** (TAL) + **F1@{10,25,50} & edit score** (TAS) +
**tight average-mAP** (spotting) | Lea 2017; SoccerNet; Soares 2022 |
605
606 Keep the internal nouns; just gloss each on first use. Low-effort, high-leverage ?
reviewers map instantly to
607 30 years of literature and see we know the failure modes.
608
609 ### 13.3 What we can / cannot claim
610
611 - ? **The integrated governance envelope as a whole** ? no found paper bundles all of:
report-first
612 contradiction detection over a *persisted temporal decision* + auditable structured
interval corrections
613 (truncate/split/drop/tighten) + **per-rule A/B lift-gating before auto-apply** +
idempotency/precedence +
614 human-in-the-loop + feeding a learned (value, confidence) scorer.
615 - ? ***"Signal not verdict" as a deliberate *design-philosophy delta*** vs. the in-domain
prior art that
616 hard-codes the same cues as if/else verdicts (state it as a philosophy delta, not a
capability they lack).
617 - ? **Warm-up/practice as a soft session-scope feature** ? thin prior art (only
patents/coaching content
618 surfaced); claim cautiously, validate from scratch.
619 - ? The **fusion mechanism** (multi-cue features ? light scorer for rally segmentation)
? ~20 yrs old in
620 racket sports (CVIU 2009; Ghosh et al. 2018 learn an SVM over per-frame features ?
smoothed rallies).
621 - ? The **persist-once/re-score substrate** as novel ? it is the *default* of the TAD
field (datasets ship
622 precomputed features so cheap heads re-train). Claim only the *idempotent/versioned
discipline*.
623 - ? The **cross-check-and-correct insight** ? Hsu et al. 2024 already cross-check
shuttle-vs-swing streams;
624 EventAnchor already does human-in-the-loop suggest-then-verify. Our delta is
*measured-before-apply +
625 auditability*, not the cross-check.
626 - ? **Static single-camera as a novel setting**, and ? the **no-regression/flag-gated
harness** as a
627 primitive (textbook MLOps) ? claim only "this discipline on a deliberately tiny golden
set."
628
629 ### 13.4 Course corrections (owner-gated; prioritized)
630
631 **Tier 1 ? protect the honesty of "measured lift" on ~6 matches (do first):**

```

632 - ****C1 ? bias-aware LOVO.**** Standard leave-one-out on ~6 imbalanced groups induces a train-vs-heldout label

633 anti-correlation that biases the estimate (a LR scored ~0.23 auROC on **random** data in the cited study).

634 Use class-balance-preserving folds (RL00CV) and ****report per-fold variance + CIs, never a single mean****, so

635 the promotion gate can't promote noise. **(Austin/Pe'er/Korem 2025, arXiv:2406.01652.)** ? ****harness/eval.****

636 - ****C2 ? calibrate each producer's confidence**** (Platt/isotonic) **before** the LR, so a CNN softmax, an audio

637 onset, and an integer net-crossing count are comparable and the over-confident cue can't swamp the arbiter.

638 **(MoCaE arXiv:2309.14976; modality competition arXiv:2203.12221.)** ? ****producers + scorer.****

639 - ****C3 ? out-of-fold (honest) stacking.**** When a producer is itself learned (shuttle-state), feed the LR its

640 **out-of-fold** outputs; extend the group-by-match split to producer training. **(Wolpert 1992.)** ?

641 ****detection?decision + harness.****

642 - ****C4 ? report the field's metrics.**** Add ****segmental F1@{10,25,50} + edit score**** and a ****tight/loose**

643 **mAP@?** sweep alongside temporal-IoU: tIoU is nearly blind to over-segmentation, so a reconciler win (or an

644 IoU-chasing rule that shreds structure) is invisible under tIoU alone. **(MS-TCN; ASRF; SoccerNet*

645 *arXiv:1804.04527; Soares 2022 arXiv:2206.07846.)** ? ****harness/eval + reconciler.****

646

647 ****Tier 2 ? architecture/discipline upgrades:****

648 - ****C5 ? resolve reconciler rules GLOBALLY**** (an activity-grammar / constraint-aware Viterbi decode), not as

649 stacked local edits with fixed precedence (which can conflict / be non-idempotent across orderings). Model

650 the structural priors (transition/duration) from data. **(Activity Grammars arXiv:2312.04266; Constraint-Aware*

651 *Decoding arXiv:2605.10149 ? read before citing.)** ? ****reconciler.****

652 - ****C6 ? brand the reconciler as "model/consistency assertions,"**** attach a per-correction confidence, and

653 route only high-confidence corrections to auto-apply, the rest to a margin-ranked human queue; version the

654 rule set as a TFDV-style owner-ratified anomaly schema. **(Kang 2020; HoloClean ~90% repair precision ? ~10%*

655 *bad; Confident Learning arXiv:1911.00068.)** ? ****reconciler + harness.****

656 - ****C7 ? a few EARLY-fusion interaction columns**** (e.g. ``shuttle_in_air AND person_two_sided``) ? a linear

657 arbiter can't represent AND/XOR; hand-crossed products recover it without leaving "signal not verdict."

658 **(Snoek 2005.)** ? ****producers + scorer.****

659 - ****C8 ? represent a missing cue as an explicit `is\_present`/uncertainty column, not a silent 0**** ? "cue

660 absent" ? "cue says no" for a linear arbiter. **(DBF arXiv:1511.03183.)** ? ****producers + scorer.****

661 - ****C9 ? keep warm-up/session-state a SOFT feature, never a hard pre-filter****, and add a court/context

662 invariance probe: the static court co-occurs with rallies, so the scorer can learn "court visible" instead

663 of "point in play" (action-context confusion). Evaluate on warm-up stretches that share the court but lack

664 point cadence. **(arXiv:2103.16155; GL-MSTCN.)** ? ****session-scope producers + scorer.****

665

666 ****Tier 3 ? leverage external resources / higher-effort:****

667 - ****C10 ? model cue CORRELATIONS**** (Snorkel/MeTaL label-model baseline; strong L1/L2): net-crossing,

668 two-sidedness, shuttle-in-air co-fire, so an unregularized LR double-counts the bundle. **(arXiv:1711.10160.)**

669 - ****C11 ? a temporal-smoothing pass**** (Kalman/HMM/Viterbi over the per-window posterior,

or an MS-TCN

670 truncated-MSE anti-fragmentation penalty) before cutting intervals ? SmartTennisTV
hits F1 ~97.5% largely

671 via Kalman-smoothing noisy per-frame predictions. *(arXiv:1801.01430; MS-TCN.)* ?
reconciler/scorer.

672 - **C12 ? pull ShuttleSet/ShuttleSet22 + CoachAI** as an external benchmark + weak-
supervision corpus to

673 attack the ~6-match bottleneck. *("Less is More" arXiv:1903.00859; ShuttleSet
arXiv:2306.04948.)* ?

674 **harness/eval + producers.**

675 - **C13 ? benchmark the rule reconciler against a learned boundary refiner** (ASRF/CBR)
to *justify* staying

676 rule-based, and ground rules in MonoTrack-style physics/cadence as **soft features,
not hard rules** (hard

677 cadence rules would re-introduce special-casing and break on doubles/lets/warm-up).
*(MonoTrack

678 arXiv:2204.01899; CBR arXiv:1705.01180.)* ? **reconciler.**

679

680 ### 13.5 Pitfalls we are actively walking into (risk register)

681

682 - **Over-segmentation hidden by tIoU** ? the dominant error mode of cheap segment
decisions; a per-window

683 scorer can look good while fragmenting rallies. ? C4.

684 - **Late-fusion miscalibration / dominant-cue swamping** (over-confident shuttle CNN,
low-discrimination

685 audio). ? C2.

686 - **LOOCV distributional bias + tiny-N variance** ? promotion gate promotes noise
without CIs. ? C1.

687 - **Stacking leakage** (in-sample learned-producer outputs) ? C3; and **threshold-
tuning-on-test** ? keep

688 *every* tunable knob (windowing, merge/NMS, regularization, reconciler thresholds)
inside the LOVO loop.

689 - **Proposal-recall ceiling** ? the decision can't recover a rally the windowing missed;
measure window

690 recall (AR@AN) separately or you'll misattribute recall loss to the scorer.

691 - **Report-first decaying into silent rewrite** ? without per-rule lift gating + a
confidence gate +

692 idempotency tests, an over-eager rule systematically truncates correct long rallies.

693 - **Static-tripod transfer gap** ? broadcast cues (camera-switch, replays, crowd audio)
that drive the

694 tennis/soccer baselines *don't exist* on our footage; don't assume their F1s transfer.

695 - **Detector domain gap inherited downstream** ? WASB/TrackNet/MonoTrack are trained on
other rigs;

696 per-frame shuttle accuracy degrades off-distribution and every cue inherits that
error. **Quantify detector

697 accuracy on our footage before trusting any cue column** (currently unmeasured).

698

699 ### 13.6 Residual gaps & honesty notes

700

701 - **Fabrications: none** across ~40 citation checks. Corrected author-lines applied
above: CVIU 2009 =

702 **Liu, Huang, Jiang, Xing, Ye, Gao** (not "Kolekar"); SmartTennisTV = **Ghosh &
Jawahar**; Sensors 2024

703 shuttle-fusion = **Hsu, Yu, Cheng**. Metadata fixes: LOOCV-bias is **Science
Advances** (10.1126/sciadv.adx6976);

704 ShuttleSet (KDD) = arXiv:2306.04948 vs ShuttleSet22 = arXiv:2306.15664 (distinct); SF-
Net evaluates on

705 THUMOS14/GTEA/BEID.

706 - **Read before citing** (verified to *exist*, not deep-read; both 2026, concurrent ?
don't void novelty but

707 are the nearest precedents): Constraint-Aware Decoding (arXiv:2605.10149, the
reconciler end-state analog)

708 and TennisExpert (arXiv:2603.13397, an audio-hits?cluster?filter rally pipeline). PCL-
Former

709 (arXiv:2507.06411) ? authorship unverified; cite without asserting authors.

```

710 - **Weakest-evidence claims (state plainly):** warm-up-as-soft-feature (novel by
*absence of counterexample*,
711 the high from-scratch-validation risk) and the measured-before-apply reconciler bundle
(also absence-based).
712 - **Not yet searched:** a recent badminton full-match frame-level rally/interruption
dataset (would relieve
713 scarcity); doubles-specific cue behavior; calibration for *count-valued* cues (net-
crossings); whether
714 RL00CV interacts badly with grouped folds at N=6 (the cited study is record-wise, not
group-wise).
715

```

15. user (2026-06-14T09:50:05.071Z)

```

1  """Segmentation-aware metrics (course-correction C4) ? make over-segmentation visible.
2
3  Temporal IoU (`metrics.score`) is nearly blind to *fragmentation*: a single rally
4  chopped into three predicted pieces, or two rallies merged across a dead-time gap, can
5  still post a moderate mean-IoU. The temporal-action-segmentation / -localization
6  literature grades those failure modes with **F1@k** (segmental F1 at several overlap
7  thresholds) and **mAP@tIoU** (ranked, confidence-aware average precision over a tIoU
8  sweep). Reporting these alongside temporal-IoU is what lets a reconciler's split/merge/
9  tighten win actually show up ? and stops an IoU-chasing rule from shredding rally
10 structure invisibly. See `docs/ANALYZERS_AND_RECONCILERS.md` §13/C4.
11
12 This is **additive**: new functions over the existing `metrics` matcher; nothing here
13 changes `score`/`compare`/LOVO behaviour. Pure (stdlib only), deterministic, no I/O
14 ?
15 so it runs in the GPU-free / numpy-free lanes too.
16
17 Conventions match `metrics`: an `Interval` is a `(start, end)` tuple of seconds.
18 A `ScoredInterval` is `(interval, confidence)` ? the confidence is whatever the
19 scorer
20 emits per predicted rally (e.g. the classifier probability), used only to *rank* for AP.
21
22 Note on the segmental *edit* score: the classic TAS edit/Levenshtein score operates on
23 the
24 ordered sequence of segment *labels* and is degenerate for a single action class (every
25 rally carries the same label), so it is intentionally omitted ? F1@k + mAP@tIoU + the
26 segment-count ratio are the over-segmentation-sensitive metrics for this binary setting.
27
28 """
29 from __future__ import annotations
30
31 from typing import Dict, List, Sequence, Tuple
32
33 from backend.eval.metrics import Interval, interval_iou, score
34
35 #: A predicted interval plus the confidence used to rank it for average precision.
36 ScoredInterval = Tuple[Interval, float]
37
38 #: Default overlap thresholds for F1@k ? the TAS convention F1@{10,25,50}.
39 DEFAULT_OVERLAPS: Tuple[float, ...] = (0.1, 0.25, 0.5)
40
41 #: Default tIoU sweep for mean average precision ? the action-detection convention.
42 DEFAULT_TIOUS: Tuple[float, ...] = (0.3, 0.4, 0.5, 0.6, 0.7)
43
44 def f1_at_overlaps(preds: List[Interval], gts: List[Interval],
45                   overlaps: Sequence[float] = DEFAULT_OVERLAPS) -> Dict[float, float]:
46     """Segmental F1 at each overlap threshold (`{overlap: f1}`) ? the TAS F1@k.
47
48     Reuses the canonical greedy one-to-one matcher, so a fragmented prediction (3 pieces
49     for 1 rally) scores extra false positives and a merged prediction misses a rally ?
50     both drag F1 down where temporal-IoU alone would not flinch.
51
52     """
53     return {ov: score(preds, gts, iou_threshold=ov).f1 for ov in overlaps}

```

```

49
50
51 def average_precision(scored_preds: Sequence[ScoreInterval], gts: List[Interval],
52                        iou_threshold: float = 0.5) -> float:
53     """Average precision at a single tIoU (all-points / VOC-2010 interpolation).
54
55     Predictions are ranked by descending confidence; each is a true positive iff it
56     claims a still-unmatched ground-truth rally at IoU >= ``iou_threshold`` (highest
57     available IoU wins, one GT per prediction). Returns AP in ``[0, 1]``. ``0.0`` when
58     there are no ground-truth rallies or no predictions reach the threshold.
59     """
60     n_gt = len(gts)
61     if n_gt == 0 or not scored_preds:
62         return 0.0
63
64     order = sorted(range(len(scored_preds)),
65                    key=lambda i: (-scored_preds[i][1], i)) # conf desc, stable by
index
66     matched_gt = set()
67     precisions: List[float] = []
68     recalls: List[float] = []
69     cum_tp = 0
70     cum_fp = 0
71     for i in order:
72         interval, _conf = scored_preds[i]
73         best_iou, best_gi = 0.0, -1
74         for gi, g in enumerate(gts):
75             if gi in matched_gt:
76                 continue
77             iou = interval_iou(interval, g)
78             if iou >= iou_threshold and iou > best_iou:
79                 best_iou, best_gi = iou, gi
80         if best_gi >= 0:
81             matched_gt.add(best_gi)
82             cum_tp += 1
83         else:
84             cum_fp += 1
85         precisions.append(cum_tp / (cum_tp + cum_fp))
86         recalls.append(cum_tp / n_gt)
87
88     # Monotone precision envelope (take the max precision to the right), then integrate
89     # over recall steps: AP = sum_k (recall_k - recall_{k-1}) * precision_env_k.
90     for i in range(len(precisions) - 2, -1, -1):
91         precisions[i] = max(precisions[i], precisions[i + 1])
92     ap = 0.0
93     prev_recall = 0.0
94     for p, r in zip(precisions, recalls):
95         if r > prev_recall:
96             ap += p * (r - prev_recall)
97             prev_recall = r
98     return ap
99
100
101 def mean_average_precision(scored_preds: Sequence[ScoreInterval], gts: List[Interval],
102                            iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> float:
103     """Mean of ``average_precision`` over a tIoU sweep (the standard mAP@tIoU)."""
104     tious = list(iou_thresholds)
105     if not tious:
106         return 0.0
107     return sum(average_precision(scored_preds, gts, t) for t in tious) / len(tious)
108
109
110 def segment_count_ratio(preds: List[Interval], gts: List[Interval]) -> float:
111     """Predicted-to-GT segment-count ratio ? a crude fragmentation diagnostic.
112

```

```

113     ``> 1`` hints over-segmentation (more predicted pieces than rallies), ``< 1`` hints
114     merging/misses. Only meaningful with ground truth present; returns ``0.0`` if
115     ``gts``
116     is empty (use F1@k / mAP for the graded picture).
117     """
118     if not gts:
119         return 0.0
120     return len(preds) / len(gts)
121
122     def segmentation_report(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
123                             overlaps: Sequence[float] = DEFAULT_OVERLAPS,
124                             iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> Dict[str,
125 object]:
126     """One-call bundle: F1@k + mAP@tIoU + per-tIoU AP + count ratio.
127
128     ``scored_preds`` carry confidences for AP; F1@k uses the bare intervals. Returns a
129     plain dict (JSON-friendly, string keys) so it can ride next to a ``compare`` result
130     or land in the experiment leaderboard.
131     """
132     preds = [iv for iv, _conf in scored_preds]
133     return {
134         "f1_at_overlaps": {f"{ov:g}": v for ov, v in f1_at_overlaps(preds, gts,
135 overlaps).items()}},
136         "ap_at_tiou": {f"{t:g}": average_precision(scored_preds, gts, t) for t in
137 iou_thresholds},
138         "mean_ap": mean_average_precision(scored_preds, gts, iou_thresholds),
139         "segment_count_ratio": segment_count_ratio(preds, gts),
140         "num_pred": len(preds),
141         "num_gt": len(gts),
142     }

```

16. assistant (2026-06-14T09:50:08.080Z)

I now have a thorough understanding. Let me check the leaderboard file format and the distill_local feature names to ground signal/group definitions, then write the design.

17. user (2026-06-14T09:50:09.495Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9     WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10     filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11     active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12     -> rally windows.
13
14     Training labels come from Gemini full-context labels via the SAME IoU matching the
15     evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16     other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17     compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18     rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20     Run:
21     python -m backend.eval.distill_local --manifest videos.json --model-out
22     output/local_rally_model.json
23     """
24     from __future__ import annotations

```

```

25     import json
26     import argparse
27     from typing import Dict, List, Optional, Tuple
28
29
30     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31     from backend.pipeline.detectors import rally_gate
32     from backend.eval.calibrate_wasb import load_golden_gts
33     from backend.eval.training import label_candidates
34     from backend.eval.classifier import LogisticRegression
35     from backend.eval.metrics import score
36     from backend.eval.gemini_refine import merge_overlaps
37
38     Interval = Tuple[float, float]
39
40     FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
41 "net_crossings"]
42     # Recall-oriented proposal: deliberately permissive so real rallies are among the
43     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
44     PROPOSAL = (0.005, 1.0, 0.5)
45     BASELINE_LOCAL = (0.02, 2.0, 2.0) # today's production-ish windowing, no learning
46
47     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
48                         proposal: Tuple[float, float, float] = PROPOSAL) ->
49 Tuple[List[Interval], List[List[float]]]:
50     """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
51 rows)."""
52     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
53     vt, mg, mwd = proposal
54     wins = TrackNetRunner.trajectory_to_action_windows(
55         points, fps=fps, frame_width=frame_width,
56         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
57     intervals = [(w["start"], w["end"]) for w in wins]
58     gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
59 crossings
60     X: List[List[float]] = []
61     for w, g in zip(wins, gated):
62         dur = max(1e-6, w["end"] - w["start"])
63         win_frames = max(1.0, dur * fps)

```

18. user (2026-06-14T09:50:10.698Z)

experiment_leaderboard.json
gemini_wrapper_2026-06-10.json
heuristic_lovo_n6.json

---LEADERBOARD---

```

[
  {
    "timestamp": "2026-06-14T06:23:51.507393+00:00",
    "iou": 0.5,
    "label_set_hash": "50d98ffbc370",
    "num_videos": 6,
    "variant_a": {
      "name": "shuttle_only",
      "spec_hash": "47c867c36f2f",
      "aggregate": {
        "precision": 0.31807603780927995,
        "recall": 0.3101947096042718,
        "f1": 0.3068611057861412,
        "mean_iou": 0.5791831889148886
      }
    },
    "variant_b": {
      "name": "shuttle_person_fusion",

```

```

    "spec_hash": "253f4709b4c4",
    "aggregate": {
      "precision": 0.3617228001510626,
      "recall": 0.2708939086387243,
      "f1": 0.2863398846047612,
      "mean_iou": 0.5690111885714347
    }
  },
  "delta": {
    "precision": 0.04364676234178266,
    "recall": -0.0393008009655475,
    "f1": -0.02052122118138,
    "mean_iou": -0.01017200034345389
  },
  "honest_delta_f1": {
    "mean_delta": -0.02052122118137999,
    "ci_low": -0.16462806145542527,
    "ci_high": 0.16224446866060574,
    "n": 6.0,
    "ci_excludes_zero": false,
    "sign_test": {
      "wins": 2.0,
      "losses": 4.0,
      "ties": 0.0,
      "n_effective": 6.0,
      "p_value": 0.6875
    }
  }
},
{
  "timestamp": "2026-06-14T06:24:56.487711+00:00",
  "iou": 0.5,
  "label_set_hash": "50d98ffbc370",
  "num_videos": 6,
  "variant_a": {
    "name": "shuttle_person",
    "spec_hash": "ee89f1e41604",
    "aggregate": {
      "precision": 0.3617228001510626,
      "recall": 0.2708939086387243,
      "f1": 0.2863398846047612,
      "mean_iou": 0.5690111885714347
    }
  }
}

```

19. assistant (2026-06-14T09:50:12.623Z)

I have everything I need. Now I'll write the full design document.

20. assistant (2026-06-14T09:52:50.807Z)

Ablation framework ? quantifying each signal's marginal contribution over time

****Status:**** DESIGN ? owner-gated, NOT started. A peer to
 [`EXPERIMENT\_HARNESS.md`](../docs/EXPERIMENT_HARNESS.md) and
 [`ANALYZERS\_AND\_RECONCILERS.md`](../docs/ANALYZERS_AND_RECONCILERS.md). Docs-only here; each
 phase below is a separate, owner-gated, default-OFF PR.

> ****One-line intent (owner).**** A *systematic, repeatable* version of the one-off ablation: run
 > it and it EXCLUDES a single signal, measures whether quality dropped, and ? tracked across the
 > EVOLUTION of quality ? tells us QUANTIFIABLY whether an old signal still earns its keep or has
 > become dead weight. The ablation table IS the empirical backbone of the paper-coauthor aim
 > ([`MEMORY` paper-coauthor-aim], honest measured results only).

0. TL;DR

The harness already computes a B?A lift honestly (LOVO + bootstrap CI + paired sign test, ``experiment.compare` + `eval_stats``). ****An ablation is just a structured family of those comparisons**** where B = "full signal set" and each A = "full set MINUS one signal" ? so the ?F1 it reports ***is*** that signal's ****marginal contribution****. We add (1) a leave-one-signal-out (LOSO) / leave-one-group-out (LOGO) ****runner**** over the golden corpus that emits a per-signal ?F1 table (CI + sign test), (2) a ****relevance ledger**** that appends each run as a timestamped row so a signal's contribution is a time series ? a signal trending to ?0 (CI spanning 0, sign test n.s. across several iterations) is flagged as candidate dead weight, (3) ****interaction handling**** (leave-PAIR-out + backward elimination, optional Shapley) so two-cheap-alone / useful-together signals aren't wrongly cut, (4) honest small-N reuse of ``eval_stats`` with a ****no-over-claim contract****, (5) integration with the leaderboard + ``QUALITY_ITERATIONS.md``, and (6) a per-user generalization ("signal X not relevant FOR THIS USER") that reuses ``per_user_gate``.

****Nothing here is new scoring math.**** It is a **driver** over ``experiment.compare` + `eval_stats` + `segmentation_metrics``, exactly as ``compare`` was a driver over ``metrics`/`calibration``.

1. What "a signal" is (the unit of ablation)

A signal = ****one feature column**** (or a named group of columns from one producer) that the scorer

consumes. The repo already treats signals as columns

(`[`ANALYZERS_AND_RECONCILERS.md`](../docs/ANALYZERS_AND_RECONCILERS.md)`)

\$1/\$4): ``experiment.predict`` reads ``feature_names`` out of persisted ``stats`` and gates/weights them.

So ****ablating a signal = removing its column(s) from a `Variant`'s `feature\_names`**** (and, for a gate-style signal like Gate-A, zeroing ``min_crossings``). No code branch; no re-detection.

The signal inventory today (the things the runner ablates), grouped by producer:

```
| Group (producer) | Signal columns | Source |
|---|---|---|
| `shuttle` | `duration`, `peak_velocity`, `mean_velocity`, `active_ratio` |
`distill_local.FEATURE_NAMES` |
| `gate_a` (net-crossings) | `net_crossings` (as a gate `min_crossings`, AND/or a column) |
`rally_gate` / `experiment_gate_mask` |
| `person` | `players_in_court`, `two_sided_motion`, `mean_player_motion`, `in_court_ratio`,
`shuttle_player_colocation` | `fusion_features.PERSON_FEATURE_NAMES` |
| `audio` | `audio_hit_count`, `audio_hit_rate`, `audio_hit_strength_mean` |
`audio_features.AUDIO_FEATURE_NAMES` |
| *(future)* `service_fault`, `warmup`, `shuttle_state` | `<producer>__<key>` (+ `confidence`)
| `ANALYZERS_AND_RECONCILERS.md` §3 |
```

The runner takes a ****signal registry****: an ordered list of ``Signal(name, group, columns, gate_knob=None)`` records. Two ablation granularities fall out of one registry:

- ****Leave-one-signal-out (LOSO)**** ? remove ONE column at a time. Fine-grained ("is ``shuttle_player_colocation`` pulling its weight?").
- ****Leave-one-GROUP-out (LOGO)**** ? remove a whole producer's columns at once ("is the entire **person** cue still relevant, or has the shuttle+audio set subsumed it?"). This is the owner's "is this old SIGNAL still relevant" question at the cue granularity, and it is what the ``QUALITY_ITERATIONS`` person-vs-audio rows are already informally doing.

> ****Naming note (don't confuse two LOGOs).**** ``eval_stats.grouped_folds`` is leave-one-VIDEO-group-out
> over the EVAL units (group by match) ? the honesty axis. This doc's LOGO is leave-one-SIGNAL-group-out
> over the FEATURE axis. Both run together: each signal-group ablation is measured **with** video-grouped
> LOVO underneath. Keep the terms "signal-LOSO/LOGO" vs "video-LOVO/LOGO" distinct in code +

tables.

2. Marginal contribution ? the core measurement

For a full signal set `S` and a signal `s`, the **marginal contribution** of `s` is:

```
?(s) = score(full set S) - score(S \ {s})          # leave-one-out drop
```

measured as **F1** (primary), with **F1@k + segment-count-ratio** alongside (C4 ? over-seg that tIoU hides). Operationally this is exactly:

```
```python
```

```
full = Variant(name="full", feature_names=ALL_COLUMNS, min_crossings=2, ...)
without_s = Variant(name=f"?{s.name}", feature_names=[c for c in ALL_COLUMNS if c not in
s.columns],
 min_crossings=(0 if s.gate_knob=="min_crossings" else 2))
```

```
res = compare(golden_videos, variant_a=without_s, variant_b=full, honest_stats=True)
```

**A = ablated (smaller), B = full ? result["honest"]["paired\_delta\_f1"] is ?(s),  
a per-video-paired F1 with bootstrap CI + exact sign test, LOVO-honest if learned**

---

Reading `compare`'s `honest` block gives, per signal, for free:

- `paired\_delta\_f1.mean\_delta` ? the marginal F1 (B?A = full?ablated);
- `paired\_delta\_f1.ci\_low/ci\_high` + `ci\_excludes\_zero` ? is the drop real at n=6?
- `paired\_delta\_f1.sign\_test` ? did removing `s` hurt on a MAJORITY of videos?
- the C4 segmentation block ? did removing `s` fragment/merge rallies even if F1 barely moved?

A signal is **earning its keep** when removing it produces a drop whose **CI excludes 0 AND** the sign test agrees ( ? the majority of videos got worse) ? the same bar `per\_user\_gate` / promotion use, applied in reverse. A signal is **candidate dead weight** when removal is a wash (`mean\_delta` ? 0, CI spans 0, sign test n.s.) **across multiple iterations** (\$4 ? one wash is noise at n=6, a *trend* is signal).

```
> ? LOSO is necessary but not sufficient. Leave-one-out attributes the unique contribution
> of `s` given everything else present. It MISSES redundancy and synergy: two signals each
useless
> alone but jointly useful both show ?0 LOSO drop (each is covered by the other ? "drop one, the
> other compensates"); and a signal can look free under LOSO yet be the linchpin of a pair. $5
adds
> leave-PAIR-out + backward elimination to recover this. Report LOSO as the screening pass,
never
> as the final verdict to delete a signal.
```

---

## 3. The runner (Phase 0)

`backend/eval/ablation.py` ? a pure driver, no new scoring math. Public surface:

```
```python
```

```
@dataclass(frozen=True)
```

```
class Signal:
```

```
    name: str                # "person.colocation" | "gate_a" | "audio"
    group: str                # "person" | "gate_a" | "audio" | "shuttle"
    columns: Tuple[str, ...]  # feature columns this signal contributes
    gate_knob: Optional[str] = None # e.g. "min_crossings" for the Gate-A threshold signal
```

```
@dataclass
```

```
class AblationConfig:
```



```

full: Variant          # the full-set reference variant (B)
signals: List[Signal]  # the inventory to ablate
granularity: str = "signal" # "signal" (LOSO) | "group" (LOGO)
iou: float = 0.5
honest: bool = True    # LOVO for learned variants
seed: int = 0          # threaded into eval_stats bootstrap (determinism)

```

```

def ablate_one(videos, cfg, signal_or_group) -> Dict:      # one row: ?(s) + CIs + sign test + C4
def run_ablation(videos, cfg) -> AblationReport:          # all signals ? a sorted table

```

```

`run_ablation` loops the inventory, builds `without_s` by **subtracting columns / zeroing gates**
from `cfg.full`, calls `compare(videos, without_s, cfg.full)`, and collects each `honest` block
into an `AblationReport`. **It reuses `compare` verbatim** ? so learned variants are LOVO-
honest,
threshold/calibration stay inside the fold (C2), and the C1+C4 honest stats come for free.

```

```

**Output table** (sorted by marginal ?F1 desc ? the contribution ranking):

```

```

...
Ablation report ? full set = {shuttle, gate_a, person, audio} . n=6 . IoU 0.5 . LOVO-
honest
signal      ?F1(full?ablated)  95% CI          sign test  CI?0  verdict
gate_a      +0.074              [+0.042, +0.104]  6W/0L p=.031  yes   EARNs KEEP
audio       +0.041              [?0.054, +0.137]  4W/2L p=.69   no    uncertain (keep,
watch)
shuttle     +0.301              [ ? ]            6W/0L          yes   EARNs KEEP (load-
bearing)
person      ?0.021              [?0.165, +0.162]  2W/4L p=.69   no    CANDIDATE DEAD
WEIGHT
...

```

(The `gate\_a`/`person` numbers above are the already-measured `QUALITY\_ITERATIONS` 2026-06-14 values, shown to anchor the format ? the runner reproduces them as a regression check on day one.)

```

**Reuses, no reinvention:** `experiment.compare`/`evaluate_variant`/`predict`,
`eval_stats.paired_delta_ci`/`paired_sign_test`/`mean_with_ci`, `segmentation_metrics`,
`record_experiment` (each ablation row is just an experiment record), `load_video_candidates`
(the DB?`VideoCandidates` bridge, owner-side). The golden corpus is the 6 matches in
`output/human_lovo_manifest.json` (`QUALITY_ITERATIONS` $"Golden corpus").

```

```

**Determinism + small-N honesty:** the bootstrap seed is threaded; n is carried on every row;
no row is allowed to print a verdict stronger than the stats support ($6 contract).

```

```

---
```

4. The Signal-Relevance Ledger (a time series of marginal contribution)

The single most novel piece for the owner's "is this OLD signal still relevant?" question. Today the leaderboard records *pairwise A/B experiments*; it does NOT track a *signal's contribution as the system evolves*. The ledger does.

```

**File:** `eval_baselines/signal_relevance_ledger.json` (tracked, baseline-sized ? same posture
as
`experiment_leaderboard.json`; NOT a service, per `EXPERIMENT_HARNESS` $9). One **run** appends
one
block; one **row per signal** inside it:

```

```

```json
{

```

```

 "timestamp": "2026-06-21T?",
 "label_set_hash": "50d98ffbc370", // ties a contribution to the exact golden labels (gt
drift guard)
 "full_set_hash": "253f4709b4c4", // spec_hash of the full Variant at this iteration
 "full_set_signals": ["shuttle","gate_a","person","audio"],
 "num_videos": 6,
 "granularity": "group",
 "rows": [
 {"signal":"gate_a","group":"gate_a","delta_f1":0.074,"ci_low":0.042,"ci_high":0.104,
 "ci_excludes_zero":true,"sign":{"wins":6,"losses":0,"p_value":0.031},
 "f1_at_overlaps_delta":{"?},"segment_count_ratio_delta":-0.67,"n":6,"verdict":"earns_keep"},
 {"signal":"person", ? ,"verdict":"candidate_dead_weight"}
]
 }
}
...

```

**\*\*Why it's a time series, not a snapshot.\*\*** As the full set evolves (a new cue lands, a label is fixed, the corpus grows), each signal's marginal contribution is RE-MEASURED *\*against the new full set\**. So ``gate_a``'s row in run-1 and run-7 are comparable points on a curve. A reader (or a tiny report tool) can ask:

- **\*\*\*"Is ``person`` still dead weight?"\*\*** ? plot ``delta_f1`` for ``person`` across runs. Persistently `?0` with CI spanning `0` ? flag ``STALE`` (candidate for removal / archival).
- **\*\*\*"Did ``gate_a`` decay when audio landed?"\*\*** ? a once-strong signal whose marginal contribution *\*shrinks\** as a newer signal subsumes it = quantified **\*\*redundancy onset\*\*** (the newer signal now carries what the old one used to). This is the dead-weight question answered with a number.
- **\*\*\*"Did corpus growth resolve audio?"\*\*** ? ``audio``'s CI should TIGHTEN as `n` grows; a contribution that was "uncertain (CI spans `0`) at `n=6`" becoming "earns keep (CI clears `0`) at `n=20`" is the honest promotion trigger ? and the inverse (still spanning `0` at large `n`) is honest demotion.

**\*\*Staleness flag (quantified dead-weight rule).\*\*** A signal is flagged ``STALE_CANDIDATE`` when, over the **\*\*last K runs\*\*** (`K?3`, configurable) at a comparable `n`: every run's ``ci_excludes_zero == false`` AND the sign test never reaches ``p ? p_threshold`` AND the rolling-mean ``delta_f1`` is within ``[??, +?]` (? small, e.g. `0.01`). That is "removal never measurably hurt, repeatedly" ? the defensible, reproducible version of "this old signal is dead weight." Flagging is **\*\*report-only\*\***; the owner decides whether to retire the column (retiring = drop it from the ``full`` Variant; rollback = add it back ? same default-OFF / config-flip discipline). **\*\*A structurally-protected signal is NEVER auto-flagged\*\*** (see §6 / ``per_user_gate`` structural rule: a guard earns its keep through failure modes the golden corpus may not contain ? e.g. Gate-A on held-shuttle FPs).

A ``tools/relevance_report.py`` (or a notebook cell) renders the ledger as a small markdown/ASCII sparkline table for ``QUALITY_ITERATIONS.md`` ? the ablation table *\*is\** the paper's empirical backbone, maintained automatically instead of by hand.

---

## 5. Redundancy & interaction ? when LOSO lies

Three named failure modes of naive leave-one-out, and the handling for each (cost-ordered cheap?dear):

Failure mode	What LOSO shows	What's true	Handler
<b>**Redundancy**</b> (two signals carry the same info)	each LOSO ? ? 0 (the other compensates)	together they're load-bearing; dropping BOTH tanks F1	<b>**leave-PAIR-out**</b> + <b>**backward elimination**</b>

| **\*\*Synergy / XOR\*\*** (useless alone, useful together, e.g. `shuttle\_in\_air AND person\_two\_sided`)  
 | each LOSO ? ? 0 (or even +) | the PAIR is the contribution; a linear arbiter can't even see it  
 | **\*\*leave-pair-out\*\*** + an **\*\*interaction column\*\*** (C7) |  
 | **\*\*Suppressor\*\*** (a signal helps only by correcting another) | LOSO ? small | removing it *\*with\**  
 its partner changes the story | **\*\*backward elimination\*\*** path-dependence note |

**\*(a) Leave-pair-out (LPO).\*** For the suspect set (signals whose LOSO ? spans 0), also run  
 `compare(full ? {s\_i, s\_j}, full)` for each pair. If a pair's joint drop's CI clears 0 while  
*\*both\**  
 singletons' did not ? **\*\*redundant-but-jointly-useful\*\***: keep both, do NOT cut either on LOSO  
 evidence. Cost:  $O(k^2)$  comparisons vs  $O(k)$  for LOSO ? bounded ( $k \geq 4$  groups ? 6 pairs), and  
 only  
 run on the LOSO-ambiguous subset, so cheap in practice. Gate it behind a flag (`--pairs`);  
 default  
 LOSO-only.

**\*(b) Backward elimination (greedy, honest).\*** The decision procedure that *\*uses\** the marginal  
 numbers: starting from the full set, repeatedly drop the signal whose removal is the **\*\*least  
 harmful\*\*** (most-positive / least-negative ?F1 with CI NOT clearing 0), re-measure the new full  
 set,  
 repeat until every remaining signal's removal measurably hurts (CI clears 0 + sign agrees). The  
 surviving set is the **\*\*honest minimal sufficient signal set\*\***. This is the principled answer to  
 "trim  
 dead weight" ? and it is path-dependent by construction (it re-measures after each drop), so it  
 catches redundancy that one-shot LOSO can't. Cost:  $O(k^2)$  worst case; report the elimination  
*\*path\**  
 (order + ? at each step) in the ledger so the trim is auditable, not a black box.

**\*(c) Shapley-style attribution (optional, owner-gated, higher cost).\*** The theoretically-fair  
 "average marginal contribution over all subset orderings" ? the only attribution that is order-  
 and  
 redundancy-fair. Exact Shapley is  $O(2^k)$  full-set evaluations; at **\*\*k $\geq$ 6 signal GROUPS** this  
 is  
 16?64 evals ? entirely affordable offline\*\* (re-scoring is cheap; detection is already  
 persisted).  
 So: **\*\*exact Shapley over GROUPS\*\*** is in scope as an opt-in (`--shapley`), Monte-Carlo Shapley  
 only if  
 k ever grows past ~10 columns. Report it as a *\*complement\** to backward elimination (Shapley says  
 "fair  
 credit"; backward elimination says "what's safely droppable"), with the same CI honesty ? each  
 subset  
 eval is itself a LOVO mean, so a Shapley value carries propagated small-N uncertainty and must  
 not be  
 over-claimed at n=6.

**\*\*Cost trade-off summary (what to run when):\*\***  
 - **\*\*Always:\*\*** LOSO + LOGO (Phase 0) ?  $O(k)$ , the screening table.  
 - **\*\*On LOSO-ambiguous signals:\*\*** leave-pair-out ( $O(k^2)$ ) on a subset) ? catches  
 redundancy/synergy.  
 - **\*\*For a trim decision:\*\*** backward elimination ( $O(k^2)$ ) ? the minimal-set procedure + audit  
 path.  
 - **\*\*For paper-grade fair credit:\*\*** exact Shapley over groups ( $O(2^k)$ ,  $k \geq 6$ ) ? opt-in.  
 - **\*\*Interaction recovery for the scorer itself:\*\*** add hand-crossed product columns (C7) so the  
 linear  
 arbiter can *\*represent\** a synergy LPO surfaced ? then that interaction is itself an ablatable  
 signal.

---

## 6. Honest small-N handling (the non-negotiable contract)

The whole point is honest verdicts at n=6, where a single mean lies. Reuse `eval\_stats` and  
**\*\*inherit its discipline\*\***; add NO new statistics. The contract every ablation row obeys:

1. **\*\*Never a bare mean.\*\*** Every ?(s) ships with a bootstrap 95% CI + the exact paired sign test (``paired_delta_ci``). A row with no CI is a bug.
2. **\*\*A signal "earns its keep" only when\*\*** removing it drops F1 with **\*\*CI excluding 0 AND sign test agreeing\*\*** (majority of videos worse, ``p ? p_threshold``) ? the ``per_user_gate`` bar, in reverse.
3. **\*\*"Dead weight" needs a TREND, not one run.\*\*** A single wash at n=6 is *\*insufficient evidence\**, not proof of irrelevance. The ``STALE_CANDIDATE`` flag (\$4) requires K?3 consistent runs ? this is the honest version of "remove it." Until then the verdict is literally ``uncertain``, never ``irrelevant``.
4. **\*\*Structural signals are exempt from auto-demotion\*\*** (mirror ``per_user_gate`` ``structural=True``): a guard whose value is in failure modes the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked ``structural`` in the registry and **\*\*never flagged dead weight on "no measured lift."\*\*** Its ledger row records the (possibly ?0) golden ? honestly but tags ``structural_protected: true``.
5. **\*\*Group by match for the eval folds\*\*** (video-LOVO via ``compare``'s honest path) ? never a held-out window from the same video; the small corpus's LOOCV anti-correlation bias (C1) is why CIs, not means, gate.
6. **\*\*Multiplicity caveat.\*\*** Ablating k signals = k simultaneous tests; at k?4 we report raw p-values but **\*\*note the family-wise risk\*\*** in the ledger header (a Holm/BH correction is a one-liner to add when k grows ? flagged, not yet applied, to avoid over-engineering at k=4).
7. **\*\*No over-claim, ever\*\*** (attribution working agreement): the runner emits ``earns_keep`` / ``uncertain`` / ``candidate_dead_weight(trend)`` / ``structural_protected`` ? it MUST NOT print "irrelevant" or "proven useless." Negative/uncertain rows are KEPT on purpose (the honest backbone).

---

## 7. Integration with the existing machinery

- **\*\*Experiment harness / leaderboard.\*\*** Each ``ablate_one`` is a ``compare`` call ? already ``record_experiment``-able. An ablation run optionally appends its pairwise rows to ``experiment_leaderboard.json`` (provenance) AND its per-signal summary to the new ``signal_relevance_ledger.json`` (the time series). Zero new persistence primitives ? both are baseline-sized JSON, the ``EXPERIMENT_HARNESS`` \$9 posture.
- **\*\*`QUALITY\_ITERATIONS.md`.** The relevance-report renderer drops an updated ablation table into the living log at each run; the prose entry (hypothesis / measured / lesson) stays human-written. The person=no-lift / audio=promising rows already there are the first two ledger points ? the runner reproduces them as a regression check.
- **\*\*Analyzers/reconcilers (`ANALYZERS\_AND\_RECONCILERS.md`).** Every new analyzer signal (``service_fault__``, ``warmup__in_warmup``, derived ``shuttle_state``) registers in the **\*\*same signal inventory\*\*** and is ablated by the same runner the moment it persists a column ? so "does this new cue earn its keep, and does it stay earning it?" is answered uniformly. A reconciler-as-Variant (\$6 of that doc) is ablatable too (full?reconciler).
- **\*\*Promotion gate.\*\*** Ablation is the *\*complement\** of promotion: promotion asks "does ADDING B beat A?"; ablation asks "does REMOVING s from the full set hurt?". Same ``compare``/``eval_stats`` engine, opposite direction. Backward elimination (\$5b) is the natural *\*de-promotion\** procedure the gate

```

lacks today.
- **CI lane.** A `run_ablation` smoke test (synthetic candidate rows, deterministic) runs in the
 GPU-free lane like the rest of the harness; a `--check-no-regression` mode asserts the day-one
 table matches the recorded `QUALITY_ITERATIONS` numbers within tolerance (drift guard on the
 math).

```

## 8. Per-user ablation (ties to the personalization gate)

The owner's "motion cue not relevant FOR THIS USER" is per-user ablation, and it composes cleanly:  
 the runner's eval folds are videos; restrict them to **\*\*one user's videos\*\*** and you get that user's per-signal marginal contribution. The decision then reuses  
 `per\_user\_gate.should\_promote\_for\_user`  
 \*in reverse\*:

```

```python

```

Per-user relevance of signal s for user U: A = U's videos with s removed, B = U's videos

```

a = evaluate_variant(U_videos, without_s).per_video_f1      # ablated
b = evaluate_variant(U_videos, full).per_video_f1          # full
dec = should_promote_for_user(a_scores=b, b_scores=a, structural=s.structural, min_n=?)

```

Read as: is REMOVING s a (per-user) win? If yes (and not structural) ? s is dead weight
personalize that user's served Variant to the smaller set. If structural ? keep regardless
 ...

This inherits all of `per\_user\_gate`'s safety: the ****min-N floor**** (refuse to personalize off 1?2 videos), the ****CI-excludes-0 + sign-test**** bar, and ? critically ? the ****structural-guard exemption**** (a user whose corpus simply lacks held-shuttle FPs must NOT get Gate-A removed; the guard stays ON regardless of that user's measured ?). Output is a ****per-user relevance ledger**** keyed by user_id, the personalization analog of \$4, feeding the same flag-gated, fall-back-to-baseline promotion path. This is owner-gated (personalization is product/strategy) and arrives after the global ledger proves out.

9. Phased, PR-sized build plan

Engineering (E) = good-engineering-fix, mergeable on verified-green CI per the merging working agreement. Owner-gated (O) = product/strategy/personalization, needs explicit approval.

****Phase 0 ? the leave-one-signal-out runner + results table (E):****

```

- **PR-A0 ? Signal inventory + `ablation.py` LOS0/LOG0 runner.** `Signal`/`AblationConfig`/
  `AblationReport` + `ablate_one` + `run_ablation`, a pure driver over `experiment.compare` (no
  new scoring math). Hard-code the current 4-group inventory; `structural` flag on `gate_a`. Unit-
  tested
  on synthetic candidate rows (deterministic, GPU-free). **DoD:** reproduces the recorded
  `QUALITY_ITERATIONS` person/gate_a numbers from a fixture within tolerance.
- **PR-A1 ? Results table + leaderboard wiring.** Sorted ASCII/markdown ablation table; append
  each
  row to `experiment_leaderboard.json` via `record_experiment`; a `--check-no-regression` mode.
  CLI
  `python -m backend.eval.ablation --manifest output/human_lovo_manifest.json`.

```

```

**Phase 1 ? the time-series relevance ledger (E for the mechanism; 0 to retire any signal):**

- **PR-L0 ? `signal_relevance_ledger.json` writer** (append one block/run; `label_set_hash` +
`full_set_hash` provenance). (E)
- **PR-L1 ? `tools/relevance_report.py`** renders the ledger as a per-signal sparkline table +
computes the `STALE_CANDIDATE` flag (K-run trend rule, §4); drops the table into
`QUALITY_ITERATIONS.md`. Report-only. (E)
- **PR-L2 ? act on a flag (retire / archive a signal).** Owner-gated: removing a column from the
`full` Variant is a product call (rollback = re-add). (0)

**Phase 2 ? interaction analysis (E for the analysis; opt-in flags):**

- **PR-I0 ? leave-pair-out** on the LOS0-ambiguous subset (`--pairs`), redundancy/synergy table.
(E)
- **PR-I1 ? backward elimination** procedure + audited elimination-path record ? minimal
sufficient
set. (E)
- **PR-I2 ? exact Shapley over groups** (`--shapley`, k?6) + C7 interaction-column support so a
surfaced synergy is itself ablatable. (E, paper-track)

**Phase 3 ? per-user ablation (0 ? personalization is product/strategy):**

- **PR-U0 ? per-user relevance runner** reusing `per_user_gate` in reverse + the structural
exemption;
per-user relevance ledger keyed by user_id; feeds the flag-gated, fall-back-to-baseline path.
(0)

```

Each PR: ONE work item, off `origin/master`, default-OFF, gated by the no-regression invariant + `run\_regression.py`. Phases 1?3 are independent of each other given Phase 0 and can be reprioritized.

10. Verification

```

- **Offline / dev-container (no GPU/torch):** `run_ablation` deterministic on synthetic
candidate
rows; the day-one table reproduces the recorded `QUALITY_ITERATIONS` person/gate_a numbers
(math
drift guard); leave-pair-out on a constructed redundant pair (both singletons span 0, the pair
clears 0) and a constructed synergy pair behave as designed; the `STALE_CANDIDATE` trend rule
fires
on a 3-run synthetic ledger and stays silent on a 1-wash ledger; structural signals never
flag;
ledger round-trips JSON with stable hashes; no row ever prints a verdict stronger than its
stats.
- **Machine-side (owner, GPU/WSL):** persist the full signal set once on the 6 golden videos
(`fusion_golden.py` substrate already exists), run `run_ablation` LOV0, confirm the per-signal
?F1
matches a manual `compare(without_s, full)` spot-check; run backward elimination and sanity-
check
the surviving set against intuition (shuttle + gate_a survive; person is the elimination
candidate).
- **Docs:** the ledger table renders into `QUALITY_ITERATIONS.md`; this plan graduates to
`docs/` and
is cross-linked from `EXPERIMENT_HARNESS.md` / `ANALYZERS_AND_RECONCILERS.md` when greenlit.

```

11. Open design questions (for the owner)

1. **Granularity default ? column (LOS0) or producer-group (LOG0)?** The owner's "is this old *signal* still relevant" reads at the GROUP level (person/audio/shuttle), but column-level catches

a single dead column inside a useful group. Default to LOGO for the relevance verdict, expose LOSO

for diagnosis? Or always report both?

2. ****Dead-weight trend threshold (K, ?, p).**** K?3 consecutive washes at comparable n, |rolling ?|<?,
 sign never significant ? are these the right knobs, and should the flag require n to be *non-decreasing* across the K runs (so corpus shrinkage can't fake staleness)?
3. ****Re-measure-on-every-iteration cost.**** The ledger's value is re-measuring every signal against
 each new full set. On the persisted golden substrate this is cheap; do we want it on EVERY harness
 run (auto), or a separate explicit `make ablate` cadence (e.g. per merged cue + weekly)?
4. ****Shapley scope.**** Exact Shapley over ?6 groups is affordable ? is paper-grade fair-credit attribution worth the extra surface now, or deferred until the corpus is large enough that its
 small-N uncertainty isn't itself the dominant caveat?
5. ****Multiplicity correction.**** Apply Holm/BH across the k simultaneous ablation tests now (more honest, slightly more conservative ? may mask a real signal at n=6), or report raw p with a family-wise caveat until k grows?
6. ****Per-user trigger.**** Per-user ablation needs ? `min\_n` videos/user ? do enough users have that,
 and is per-user signal-retirement (vs only per-user signal-*addition*) even desired, or should
 personalization stay add-only for safety?

21. user (2026-06-14T09:52:50.833Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\ABLATION_FRAMEWORK_PLAN.md (file state is current in your context ? no need to Read it back)

22. assistant (2026-06-14T09:53:13.544Z)

Design written to `scratch/ABLATION\_FRAMEWORK\_PLAN.md`. Below is the requested return: exec summary, the Phase-0 PR list, and the top open design questions.

Exec summary (~250 words)

The harness already measures a B?A lift honestly ? LOVO + bootstrap CI + paired sign test (`experiment.compare` reading from `eval\_stats`). ****An ablation is just a structured family of those comparisons**** where B = the full signal set and each A = "full set MINUS one signal," so the ?F1 it reports IS that signal's marginal contribution. No new scoring math ? the ablation layer is a *driver* over `compare` + `eval\_stats` + `segmentation\_metrics`, exactly as `compare` was a driver over `metrics`/`calibration`.

A signal = one feature column (or a producer's column group: `shuttle`, `gate\_a`, `person`, `audio`, plus future `service\_fault`/`warmup`/`shuttle\_state`). Ablating it = subtracting its columns from a `Variant`'s `feature\_names` (or zeroing `min\_crossings` for Gate-A) ? no branch, no re-detection. The runner emits a per-signal ?F1 table (CI + sign test + C4 over-seg). The ****Signal-Relevance Ledger**** appends each run as a timestamped block so a signal's contribution becomes a time series; a signal trending to ?0 across K?3 runs (CI spanning 0, sign-test n.s.) is flagged `STALE\_CANDIDATE` ? the quantified "is this old signal dead weight?" answer.

Because naive leave-one-out misses redundancy/synergy, the design adds leave-pair-out + greedy backward-elimination (minimal sufficient set) and optional exact Shapley over groups (k?6, affordable). Small-N honesty is inherited wholesale from `eval\_stats` + `per\_user\_gate`'s bars, including the ****structural-guard exemption**** (Gate-A never auto-demoted on "no golden lift"). It composes with the leaderboard, `QUALITY\_ITERATIONS.md`, and generalizes to per-user ablation via `per\_user\_gate` in reverse ("motion cue not relevant FOR THIS USER").

Phase-0 PR list (engineering, mergeable on green CI)

- **PR-A0** ? Signal inventory + `ablation.py` LOSO/LOGO runner. **Signal** / `AblationConfig` / `AblationReport` + `ablate\_one` + `run\_ablation`, a pure driver over `experiment.compare` (no new scoring math). Hard-code the current 4-group inventory; `structural` flag on `gate\_a`. Unit-tested on synthetic candidate rows (deterministic, GPU-free). **DoD:** reproduces the recorded `QUALITY\_ITERATIONS` person/gate_a numbers from a fixture within tolerance.
- **PR-A1** ? Results table + leaderboard wiring. **Sorted** markdown/ASCII ablation table; append each row to `experiment\_leaderboard.json` via `record\_experiment`; a `--check-no-regression` mode (math drift guard). CLI: `python -m backend.eval.ablation --manifest output/human\_lovo\_manifest.json`.

(Later phases ? time-series relevance ledger, interaction analysis, per-user ablation ? are listed and sequenced in §9, with engineering vs owner-gated marked.)

Top open design questions

1. **Granularity default** ? report relevance at producer-GROUP level (LOGO, matches "is this old signal relevant") or column level (LOSO, catches a dead column inside a useful group), or always both?
2. **Dead-weight trend rule** ? are K?3 consecutive washes / |rolling ?|<? / sign-never-significant the right knobs, and should the flag require n non-decreasing across the K runs so corpus shrinkage can't fake staleness?
3. **Re-measure cadence** ? re-run the ledger on EVERY harness run (auto) or a separate explicit `make ablate` cadence (per merged cue + periodic)?
4. **Shapley scope** ? exact Shapley over ?6 groups is affordable; worth adding now for paper-grade fair credit, or defer until the corpus is large enough that its small-N uncertainty isn't the dominant caveat?
5. **Multiplicity correction** ? apply Holm/BH across the k simultaneous ablation tests now (more honest, may mask a real signal at n=6) or report raw p with a family-wise caveat until k grows?
6. **Per-user trigger** ? do enough users clear `min\_n` videos, and is per-user signal-*retirement* desired at all, or should personalization stay add-only for safety?

Key grounding files read: `backend/eval/experiment.py`, `backend/eval/eval\_stats.py`, `backend/eval/per\_user\_gate.py`, `backend/eval/segmentation\_metrics.py`, `backend/pipeline/detectors/fusion\_features.py`, `backend/pipeline/detectors/audio\_features.py`, `backend/eval/distill\_local.py` (signal/group inventory), `docs/EXPERIMENT\_HARNESS.md`, `docs/ANALYZERS\_AND\_RECONCILERS.md`, `docs/QUALITY\_ITERATIONS.md`, and `eval\_baselines/experiment\_leaderboard.json` (the honest-block + record shape the ledger extends).